



FACULTY OF COMPUTING
UNIVERSITY OF
SRI JAYEWARDENEPURA

ShortStay – Online Marketplace for Short - Term Accommodations

W.P.Y.G. DE SILVA

GROUP - 02

Registration number: FC110986

Index number: FC213039

Name of the Supervisor:

Mr. K.D. Tharindu Rangana

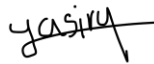
July 2026

**This dissertation is submitted in partial fulfillment of the requirement of the Degree of
Bachelor of Computing (Honors) in Information Systems of the
Faculty of Computing, University of Sri Jayewardenepura.**

Declaration

I certify that this dissertation does not incorporate, without acknowledgment, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text. I also hereby give consent for my dissertation, if accepted, to be made available for photocopying and for interlibrary loans, and for the title and abstract to be made available to outside organizations.

Signature of Candidate :



Date : 10/07/2026

Name of Candidate: W.P.Y.G. De Silva

Countersigned by:

Signature of Supervisor(s)/Advisor(s): Date : 10/07/2026

Name(s) of Supervisor(s)/Advisor(s): Mr. K. D. Tharindu Rangana

Abstract

The rapid growth of tourism and the increasing demand for short-term accommodations have created new opportunities for property owners and travelers. However, many local property owners face challenges when using existing international accommodation platforms due to complexity, limited local support, and the lack of proper verification mechanisms. Therefore, this project introduces ShortStay, a web-based online marketplace designed to provide a simple, reliable, and locally focused platform for short-term accommodation services.

The main objective of ShortStay is to connect guests with verified hosts while providing a secure and efficient accommodation booking experience. The system consists of five main user roles: Guest, Host, Admin, Accountant, and Verifier. Guests can search, filter, and book available accommodations, while hosts can manage property listings and booking requests. The Admin manages overall system operations, the Accountant manages financial transactions, payments, and related records, while the Verifier inspects properties and ensures that accommodations meet the required quality standards before approval.

The system was developed using an agile software development approach to support iterative development and continuous improvement. The frontend was implemented using React.js, the backend was developed using Node.js with Express.js, and MySQL was used as the database management system. UML diagrams, database design, and role-based architecture were used to ensure system clarity, scalability, and maintainability.

The completed ShortStay system provides essential functionalities including property management, accommodation booking, payment management, property verification, and user review mechanisms. The platform helps local property owners generate additional income opportunities while providing guests with a trusted and convenient accommodation booking solution.

Keywords: Short-term accommodation, Online marketplace, Property management, Booking system, Verification system, Web application

Acknowledgment

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of our project, “ShortStay – Online Marketplace for Short-Term Accommodations.”

First and foremost, we would like to thank our project supervisor, Mr. K.D. Tharindu Rangana, for the valuable guidance, continuous support, and constructive feedback provided throughout the project. The advice and recommendations helped us improve our project idea, overcome challenges, and successfully complete this work.

We would also like to extend our appreciation to the lecturers and staff members of the Faculty of Computing, University of Sri Jayewardenepura, for providing the knowledge, resources, and academic environment necessary to complete this project successfully.

Our sincere appreciation goes to all team members for their commitment, cooperation, and teamwork throughout the requirement analysis, system design, development, testing, and documentation phases. The contribution of each member in their assigned responsibilities played an important role in achieving the objectives of this project.

Finally, we would like to thank our families, friends, and everyone who supported us with encouragement and assistance throughout the completion of this project.

Table of Contents

Declaration.....	2
Abstract.....	3
Acknowledgment	4
Table of Contents.....	5
Table of Figures	11
List of Tables	13
List of Acronyms	14
Chapter 1 - INTRODUCTION	15
1.1 Motivation for the project	15
1.2 Objectives	15
Main Objective.....	15
Specific Objectives	15
1.3 Scope of the project	16
1.4 Outline of the remaining chapters.....	17
Chapter 2 - ANALYSIS.....	19
2.1 Current Manual method	19
2.2 Drawbacks of Existing method.....	19
Lack of Centralized Platform.....	20
Limited Property Verification.....	20
Manual Booking Management.....	20
Payment Management Issues	20
Limited Trust Between Users	20
Challenges for Local Users	20
2.3 Review of Similar systems.....	20
Airbnb	21

Main Features:	21
Limitations:	21
Booking.com	21
Main Features:	21
Limitations:	21
Agoda	22
Main Features:	22
Limitations:	22
Comparison with ShortStay	22
2.4.1 Functional requirements.....	22
Guest Requirements	23
Host Requirements.....	23
Admin Requirements	23
Accountant Requirements.....	23
Verifier Requirements.....	24
2.4.2 Non-functional requirements	24
Security	24
Usability.....	24
Performance	24
Reliability.....	24
Maintainability.....	24
Scalability	24
2.5 Process Model.....	24
Sprint 1 – User Authentication and User Management	25
Sprint 2 – Property Management Module.....	25
Sprint 3 – Accommodation Booking Module.....	25
Sprint 4 – Payment and Property Verification Modules.....	26

Sprint 5 – System Integration, Review Management and Final Testing	26
Chapter 3 - DESIGN.....	27
3.1 Design strategies	27
1. Modular Design	27
2. Role-Based Design.....	27
3. Database-Centered Design.....	28
4. Responsive User Interface Design	28
3.2 Justification for the selected design strategy	28
3.3 Architectural design	29
Presentation Layer (Frontend)	29
Business Logic Layer (Backend)	30
Data Access Layer (Database)	30
3.4 Object-Oriented Methodical approach.....	31
User	31
Guest	31
Host	31
Property	32
Booking	32
Payment	33
Verification	33
3.5 High-Level Use Case Diagram for the Proposed System.....	33
Guest	34
Host	34
Admin	34
Accountant	34
Verifier	34
3.6 ER Diagram for the Proposed System	35

3.7 Class Diagram for the Proposed System.....	37
3.8 User Interface Designing for the System	39
Guest Interface	40
Host Interface	42
Admin Interface	43
Accountant Interface	45
Verifier Interface	46
Chapter 4 – IMPLEMENTATION	48
4.1 Introduction.....	48
4.2 Implementation Environment	48
4.3 System Developed Tools and Technologies	49
4.4 Major Code Segments.....	51
4.4.1 Database Connection	51
4.4.2 Server Initialization and API Route Registration	52
4.4.3 Login, Account Checks and JWT Generation	53
4.4.4 JWT Authentication Middleware.....	54
4.4.5 Frontend Role-Based Route Protection	55
4.4.6 Frontend API Communication.....	56
4.4.7 Booking Creation and Availability Validation	57
4.4.8 Payment Intent Creation	57
4.5 System module structure.....	58
4.6 Re-usable Components	59
Chapter 5 – EVALUATION.....	61
5.1 Introduction.....	61
5.2 System Testing.....	61
5.3 Testing Techniques	61
5.4 Testing Methodologies.....	62

5.5 System Test Plan.....	63
5.6 System Test Cases.....	64
5.7 User acceptance test.....	69
5.8 User Feedback.....	70
5.9 Limitations of Evaluation	71
Chapter 6 – CONCLUSION.....	72
6.1 Introduction.....	72
6.2 Problems Encountered	72
6.3 Future Improvements	73
6.4 Lessons Learnt	74
References.....	76
Appendix A - System Manual.....	77
A.1 Introduction.....	77
A.2 System Requirements	77
Hardware Requirements	77
Software Requirements	78
A.3 Installation Procedure	78
A.4 Database Configuration.....	79
A.5 Project Structure.....	79
A.6 Troubleshooting	80
Appendix B - Design Documentation.....	81
B.1 Introduction	81
B.2 System Architecture.....	81
B.3 Use Case Diagram	81
B.4 Class Diagram.....	82
B.5 Entity Relationship Diagram.....	83
Appendix C - User Manual.....	85

C.1 Introduction	85
C.2 User Registration	85
C.3 User Login	86
C.4 Searching for Properties	87
C.5 Booking Accommodation	89
C.6 Making Payments	91
C.7 Host Functions	92
C.8 Administrator Functions	95
C.9 Payment Manager Functions	98
C.10 Field Inspector Functions	100
C.11 User Logout	102
Appendix D - Test Results	105
Glossary	109
Index	110

Table of Figures

Figure 1 - Use Case Diagram.....	35
Figure 2 - ER Diagram.....	37
Figure 3 - Class Diagram	39
Figure 4 - User Interface for the Login Page	40
Figure 5 - User Interface for the Property search page	41
Figure 6 - User Interface for the Property details page.....	41
Figure 7 - User Interface for the Booking page	42
Figure 8 - User Interface for the Host dashboard	42
Figure 9 - User Interface for the Property management page.....	43
Figure 10 - User Interface for the Admin dashboard.....	43
Figure 11 - User Interface for the Booking management	44
Figure 12 - User Interface for the Property management	44
Figure 13 - User Interface for the Report Generation.....	45
Figure 14 - User Interface for the Payment dashboard	45
Figure 15 - User Interface for the Transaction management	46
Figure 16 - User Interface for the Verification dashboard.....	46
Figure 17 - User Interface for the Property inspection interface	47
Figure 18 - System module folder structure of ShortStay	51
Figure 19 - Code segment for database connection	52
Figure 20 - Code segment for Express server and route registration.....	53
Figure 21 - Code segment for login and JWT generation.....	54
Figure 22 - Code segment for JWT authentication middleware	55
Figure 23 - Code segment for frontend protected routes	56
Figure 24 - Code segment for Axios API helper	56
Figure 25 - Code segment for booking creation and availability validation.....	57
Figure 26 - Code segment for Stripe Payment Intent creation.....	58
Figure 27 - Use Case Diagram.....	82
Figure 28 - ER Diagram.....	84
Figure 29 - UI for User Registration.....	85
Figure 30 - UI for User Login.....	86
Figure 31 - UI for Host dashboard.....	87

Figure 32 - UI 1 for Searching for Properties	88
Figure 33 - UI 2 for Searching for Properties	89
Figure 34 - UI 1 for Booking Accommodation	89
Figure 35 - UI 2 for Booking Accommodation	90
Figure 36 - UI 3 for Booking Accommodation	90
Figure 37 - UI 4 for Booking Accommodation	91
Figure 38 - UI 1 for Making Payments	91
Figure 39 - UI 2 for Making Payments	92
Figure 40 - UI 1 for Host Functions.....	93
Figure 41 - UI 2 for Host Functions.....	93
Figure 42 - UI 3 for Host Functions.....	94
Figure 43 - UI 4 for Host Functions.....	94
Figure 44 - UI 5 for Host Functions.....	95
Figure 45 - UI 1 for Administrator Functions.....	96
Figure 46 - UI 2 for Administrator Functions.....	96
Figure 47 - UI 3 for Administrator Functions.....	97
Figure 48 - UI 4 for Administrator Functions.....	97
Figure 49 - UI 5 for Administrator Functions.....	98
Figure 50 - UI 1 for Payment Manager Functions	99
Figure 51 - UI 2 for Payment Manager Functions	99
Figure 52 - UI 3 for Payment Manager Functions	100
Figure 53 - UI 4 for Payment Manager Functions	100
Figure 54 - UI 1 for Field Inspector Functions	101
Figure 55 - UI 2 for Field Inspector Functions	101
Figure 56 - UI 3 for Field Inspector Functions	102
Figure 57 - UI 4 for Field Inspector Functions	102
Figure 58 - UI 1 for User Logout.....	103
Figure 59 - UI 2 for User Logout.....	104

List of Tables

Table 1 - Development Environment Requirements	48
Table 2 - System Development Tools and Technologies	49
Table 3 - System Module Structure	58
Table 4 - Reusable Components and Utilities.....	60
Table 5 - Testing Techniques.....	61
Table 6 - Testing Methodologies	62
Table 7 - System Test Plan	63
Table 8 - Test Cases for Authentication	64
Table 9 - Test Cases for Property and Search Management.....	65
Table 10 - Test Cases for Booking Management.....	66
Table 11 - Test Cases for Payment, Reports and Feedback.....	67
Table 12 - Test Cases for Inspection, Admin and Security	68
Table 13 - User Acceptance Test Summary	69
Table 14 - User Feedback Summary.....	70
Table 15 - Hardware Requirements	77
Table 16 - Software Requirements	78
Table 17 - Troubleshooting.....	80
Table 18 - Test Results for Authentication	105
Table 19 - Test Results for Property and Search Management	106
Table 20 - Test Results for Booking Management	107
Table 21 - Test Results for Payment, Reports and Feedback	107
Table 22 - Test Results for Inspection, Admin and Security	108

List of Acronyms

Acronym	Description
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
CSS	Cascading Style Sheets
ER	Entity Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JWT	JSON Web Token
JSON	JavaScript Object Notation
MVC	Model–View–Controller
MySQL	My Structured Query Language (Relational Database Management System)
OOD	Object-Oriented Design
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UAT	User Acceptance Testing
UX	User Experience

Chapter 1 - INTRODUCTION

1.1 Motivation for the project

The tourism industry in Sri Lanka has shown continuous growth over the past decade, contributing significantly to the national economy. With increasing tourist arrivals and domestic travel demand, the need for short-term accommodation services has also increased. Many property owners, especially in local areas, are now interested in renting their spaces to earn additional income.

However, existing global platforms such as Airbnb and Booking.com present several challenges for local users. These platforms are often complex, less localized, and may include high service charges. In addition, property verification processes are not always transparent for users, which can reduce trust between hosts and guests.

Furthermore, many local property owners face difficulties in managing online listings due to lack of technical knowledge and platform complexity. On the other hand, travelers often struggle to find affordable, verified, and reliable accommodations through a simple system tailored to local needs.

To address these challenges, the ShortStay system is introduced as a web-based online marketplace for short-term accommodations. It aims to provide a simple, efficient, and trustworthy platform where guests can easily find accommodations and hosts can manage their properties effectively. The system also introduces structured roles such as Accountant and Verifier to improve financial transparency and property trustworthiness, which are often limited in existing systems.

Therefore, ShortStay is developed to bridge the gap between property owners and travelers by providing a localized, user-friendly, and secure accommodation booking platform.

1.2 Objectives

Main Objective

The main objective of this project is to develop a web-based short-term accommodation marketplace that enables secure interaction between guests and hosts while ensuring reliable payment handling and property verification.

Specific Objectives

- To develop a web platform for searching and booking short-term accommodations
- To allow hosts to create, manage, and update property listings

- To implement a secure authentication system for all users
- To introduce a Verifier role for property inspection and approval
- To introduce an Accountant role for managing payments, transactions, and financial records
- To enable administrators to manage users and overall system operations
- To improve trust and transparency in the short-term rental process
- To provide an easy-to-use interface suitable for local users and tourists

1.3 Scope of the project

The ShortStay system is designed as a web-based application that supports multiple user roles and core functionalities related to short-term accommodation management.

Guest

- User registration and login
- Searching and filtering accommodations
- Viewing property details
- Booking accommodations
- Making payments
- Writing reviews and ratings
- Viewing booking history

Host

- Property registration and listing creation
- Managing property details (price, images, availability)
- Managing booking requests
- Viewing earnings and booking status

Admin

- Managing all system users
- Approving or rejecting property listings
- Monitoring system activities
- Handling user complaints and system control

Accountant

- Managing payment transactions
- Handling host payouts
- Monitoring financial records
- Processing refunds and payment verification

Verifier

- Conducting property inspections
- Verifying property quality and safety standards

- Approving or rejecting property verification requests
- Maintaining inspection records

1.4 Outline of the remaining chapters

This report consists of six chapters that describe the complete development process of the ShortStay – Online Marketplace for Short-Term Accommodations system.

Chapter 2 – Analysis

This chapter discusses the analysis phase of the ShortStay system. It explains the requirement gathering techniques used, analysis of existing accommodation platforms, identified limitations, and the requirements of the proposed system. Furthermore, relevant system analysis diagrams such as use case diagrams, class diagrams, sequence diagrams, and entity relationship diagrams are presented to provide a clear understanding of the system functionality and structure.

Chapter 3 – Design

This chapter presents the design phase of the proposed system based on the identified requirements. It describes the overall system architecture, database design, user interface design, and design approaches used during the development process. Relevant design diagrams and interface designs are included to demonstrate the structure and behavior of the ShortStay system.

Chapter 4 – Implementation

This chapter explains the implementation process of the ShortStay system. It discusses the development environment, technologies, frameworks, and tools used for implementation. The implementation of major system modules, including user management, property management, booking management, payment management, and property verification modules, are also described.

Chapter 5 – Evaluation

This chapter focuses on the testing and evaluation process carried out to ensure the quality and reliability of the ShortStay system. It explains the testing strategies, test cases, testing results, and user acceptance testing performed to verify whether the implemented system satisfies the identified requirements.

Chapter 6 – Conclusion

This chapter provides the overall conclusion of the project by discussing the achievements, challenges encountered during development, lessons learned, and possible future improvements that can enhance the ShortStay system.

Chapter 2 - ANALYSIS

2.1 Current Manual method

Currently, many short-term accommodation arrangements in Sri Lanka are handled through informal and semi-digital methods. Property owners commonly advertise their accommodations through social media platforms, personal websites, online advertisements, or by sharing information among friends and communities. Guests who are interested in renting these properties usually contact property owners directly through phone calls or messaging applications to inquire about availability, pricing, facilities, and booking details.

The booking process often requires manual communication between guests and property owners. Guests must request available dates, negotiate prices, confirm accommodation details, and make payments through separately arranged methods such as bank transfers or cash payments. Property owners manually maintain booking records and update availability based on received requests.

Although some international platforms provide online accommodation booking services, many local property owners still depend on manual processes due to platform complexity, service charges, and lack of localized support. As a result, there is a requirement for a centralized system that simplifies accommodation management while improving trust between guests and hosts.

The current manual accommodation process can be summarized as follows:

1. The property owner advertises accommodation details.
2. Guest searches for available accommodations.
3. Guest contacts the property owner.
4. The owner provides details and confirms availability.
5. Guests complete payment through an agreed method.
6. The owner manually records the booking.

This process creates difficulties in managing accommodation availability, verifying properties, and maintaining transparent communication.

2.2 Drawbacks of Existing method

The current manual method of managing short-term accommodations has several limitations that affect both guests and property owners.

Lack of Centralized Platform

Since accommodation details are distributed across different sources such as social media pages and advertisements, guests must spend additional time searching for suitable properties.

Limited Property Verification

Many existing informal methods do not include a proper property verification process. Guests may face uncertainty regarding property quality, safety, and the accuracy of provided information.

Manual Booking Management

Property owners often manage reservations manually, which can lead to:

- Double bookings
- Incorrect availability information
- Difficulty tracking booking history

Payment Management Issues

Payments are commonly handled separately between guests and hosts. This creates difficulties in tracking transactions, handling refunds, and maintaining payment records.

Limited Trust Between Users

Without a proper review, rating, and verification mechanism, guests may hesitate to book unfamiliar accommodations.

Challenges for Local Users

International platforms may include complex procedures, high service fees, and limited consideration for local requirements, making them less convenient for some local hosts.

Therefore, a localized online platform is required to overcome these limitations and provide a more reliable accommodation management solution.

2.3 Review of Similar systems

Several existing online accommodation platforms were analyzed to identify their features, advantages, and limitations.

Airbnb

Airbnb is a global accommodation marketplace that connects hosts with guests. It allows property owners to publish listings and enables guests to search and reserve accommodations.

Main Features:

- Property listing management
- Search and filtering
- Online booking
- Reviews and ratings
- Payment handling

Limitations:

- High service charges
- Less focused on local requirements
- Complex processes for some new users
- Verification process is not directly controlled by local communities

Booking.com

Booking.com provides online reservation services for hotels and private accommodations worldwide.

Main Features:

- Large accommodation database
- Online reservations
- Customer reviews
- Price comparison

Limitations:

- Mainly focused on established accommodation providers
- Limited opportunities for small local property owners
- Less flexibility for local accommodation management

Agoda

Agoda is another global travel booking platform that provides accommodation reservations.

Main Features:

- Accommodation search
- Online booking
- Promotions and discounts
- Customer reviews

Limitations:

- Primarily targets commercial accommodation providers
- Limited support for small-scale local hosts

Comparison with ShortStay

ShortStay is designed to overcome these limitations by providing a locally focused accommodation marketplace.

ShortStay introduces:

- Localized user-friendly experience
- Property verification through the Verifier
- Payment monitoring through the Accountant
- Direct support for local hosts
- Role-based management through Admin control

2.4 Requirements of the system

The requirements of the ShortStay system were identified through analysis of the current accommodation process, user needs, and limitations of existing systems.

The system requirements are categorized into:

- Functional Requirements
- Non-functional Requirements

2.4.1 Functional requirements

Functional requirements define the services and features that the ShortStay system provides to users.

Guest Requirements

The system shall allow guests to:

- Register and log into the system
- Search accommodations
- Filter properties by location, price, and availability
- View property details
- Book accommodations
- Make payments
- View booking history
- Submit reviews and ratings

Host Requirements

The system shall allow hosts to:

- Create and manage property listings
- Upload property information and images
- Update availability and pricing
- Receive booking requests
- Manage reservations
- View earnings

Admin Requirements

The system shall allow administrators to:

- Manage all users
- Monitor system activities
- Approve or remove listings
- Manage complaints
- Generate reports

Accountant Requirements

The system shall allow accountants to:

- Monitor transactions
- Manage payment records
- Handle refunds
- Track host payments
- Generate financial reports

Verifier Requirements

The system shall allow verifiers to:

- View verification requests
- Inspect property details
- Submit verification results
- Approve or reject properties
- Maintain inspection records

2.4.2 Non-functional requirements

Security

The system should protect user information through secure authentication and authorization mechanisms.

Usability

The system should provide a simple and user-friendly interface for different user groups.

Performance

The system should respond quickly and efficiently handle user requests.

Reliability

The system should provide consistent service availability with minimal failures.

Maintainability

The system should be easy to update and modify in the future.

Scalability

The system should support increasing numbers of users and properties.

2.5 Process Model

The ShortStay system was developed using the Agile software development methodology. Agile was selected because it supports iterative and incremental software development, allowing the development team to implement the system in small, manageable phases while continuously improving the product throughout the project lifecycle. Unlike traditional development approaches, Agile provides flexibility to accommodate changes in requirements and enables regular evaluation of the system as new features are developed.

The project was divided into several development sprints. Each sprint focused on implementing a specific set of system functionalities, followed by testing and improvements before proceeding to the next sprint. This iterative approach allowed the team to identify issues early, improve system quality, and ensure that each completed module could be integrated successfully with the rest of the system.

The development process of the ShortStay system consisted of the following sprints.

Sprint 1 – User Authentication and User Management

The first sprint focused on developing the core user management functionalities required for all system users.

Activities performed:

- Designed the login and registration interfaces.
- Developed user registration and login functionalities.
- Implemented role-based access for Guest, Host, Admin, Accountant, and Verifier.
- Tested authentication and user management features.

Sprint Deliverable:

A functional user authentication module with secure registration, login, and role-based access.

Sprint 2 – Property Management Module

The second sprint focused on implementing accommodation management features for hosts.

Activities performed:

- Developed property listing creation and management.
- Implemented property image uploading.
- Developed property availability management.
- Tested property management functionalities.

Sprint Deliverable:

A functional property management module allowing hosts to manage accommodation listings.

Sprint 3 – Accommodation Booking Module

The third sprint focused on implementing the booking process for guests.

Activities performed:

- Developed property search and filtering.
- Implemented booking request and confirmation functionalities.
- Developed booking history management.

- Tested booking-related features.

Sprint Deliverable:

A complete accommodation booking module enabling guests to search and reserve properties.

Sprint 4 – Payment and Property Verification Modules

The fourth sprint focused on implementing financial management and property verification features.

Activities performed:

- Developed payment transaction management.
- Developed the Accountant module for monitoring financial transactions.
- Developed the Verifier module for property inspection and approval.
- Integrated these modules with the existing system.
- Tested payment and verification functionalities.

Sprint Deliverable:

Integrated payment management and property verification modules.

Sprint 5 – System Integration, Review Management and Final Testing

The final sprint focused on integrating all developed modules and preparing the system for completion.

Activities performed:

- Developed the review and rating functionality.
- Integrated all frontend and backend modules.
- Performed system testing.
- Fixed identified issues and improved system usability.
- Prepared the project documentation.

Sprint Deliverable:

A fully integrated and tested ShortStay system ready for final submission.

Chapter 3 - DESIGN

3.1 Design strategies

The design phase of the ShortStay system focuses on transforming the identified requirements into a structured and practical system design. The main objective of the design process is to create a system that is reliable, maintainable, scalable, and user-friendly.

The ShortStay system follows a combination of Object-Oriented Design (OOD) principles and a three-tier architecture approach. The system design separates the user interface, business logic, and data management layers to improve maintainability and future expansion.

The main design strategies used for the development of ShortStay are:

1. Modular Design

The system is divided into independent modules based on user roles and functionalities. Each module handles a specific responsibility.

The major modules include:

- User Management Module
- Property Management Module
- Booking Management Module
- Payment Management Module
- Property Verification Module
- Review and Rating Module

This approach makes the system easier to develop, test, and maintain.

2. Role-Based Design

Since ShortStay supports five different user categories, the system was designed with role-based access control.

The user roles are:

- Guest
- Host
- Admin
- Accountant
- Verifier

Each role receives access only to the functionalities relevant to their responsibilities.

3. Database-Centered Design

A structured relational database design was created to manage system information effectively.

The database stores:

- User information
- Property details
- Booking records
- Payment information
- Verification details
- Reviews

Entity Relationship (ER) modeling was used to identify relationships between system entities.

4. Responsive User Interface Design

The user interface was designed with usability as a priority. The system provides simple navigation and responsive layouts to support different types of users.

The interface design focuses on:

- Easy navigation
- Clear information presentation
- Consistent design components
- User-friendly interactions

3.2 Justification for the selected design strategy

The selected design strategies were chosen to ensure that the ShortStay system can efficiently satisfy user requirements and support future improvements.

The modular design approach allows developers to work on different system components independently. Since ShortStay contains multiple functional areas such as

booking, payments, and verification, separating these modules reduces complexity and improves development efficiency.

The role-based design approach was selected because the system contains different users with different responsibilities. For example, guests should manage bookings, while verifiers should only access property inspection-related functions. This improves security and prevents unauthorized access.

The three-tier architecture approach was selected because it provides a clear separation between the frontend, backend, and database layers. This makes the system easier to maintain, test, and scale.

The object-oriented approach was selected because the system consists of real-world entities such as users, properties, bookings, and payments. Representing these entities as objects improves system organization and code reusability.

3.3 Architectural design

The ShortStay system follows a three-tier client-server architecture consisting of:

1. Presentation Layer
2. Business Logic Layer
3. Data Access Layer

Presentation Layer (Frontend)

The presentation layer is responsible for user interaction with the system.

Technology used:

- React.js

Responsibilities:

- Display user interfaces
- Handle user interactions
- Send requests to backend services
- Display system responses

Examples:

- Guest dashboard
- Host dashboard
- Admin dashboard
- Accountant dashboard
- Verifier dashboard

Business Logic Layer (Backend)

The business logic layer handles system operations and application rules.

Technology used:

- Node.js
- Express.js

Responsibilities:

- Process user requests
- Apply business rules
- Manage authentication
- Handle booking operations
- Manage verification workflows

Examples:

- Booking validation
- Property approval logic
- Payment processing logic

Data Access Layer (Database)

The data layer stores and manages system information.

Technology used:

- MySQL

Responsibilities:

- Store user details
- Store accommodation information
- Maintain booking records
- Store payment transactions
- Store verification results

The overall architecture ensures clear separation between components and improves system scalability.

3.4 Object-Oriented Methodical approach

The ShortStay system follows an Object-Oriented Design approach where system components are represented as objects with attributes and behaviors.

The main objects identified in the system are:

User

Attributes:

- User ID
- Name
- Email
- Password
- Role

Behaviors:

- Login()
- Logout()
- UpdateProfile()

Guest

Attributes:

- Guest ID

Behaviors:

- SearchProperty()
- CreateBooking()
- SubmitReview()

Host

Attributes:

- Host ID

Behaviors:

- AddProperty()
- UpdateProperty()
- ManageBooking()

Property

Attributes:

- Property ID
- Location
- Price
- Availability
- Description

Behaviors:

- UpdateDetails()
- ChangeAvailability()

Booking

Attributes:

- Booking ID
- Booking Date
- Status

Behaviors:

- ConfirmBooking()
- CancelBooking()

Payment

Attributes:

- Payment ID
- Amount
- Payment Status

Behaviors:

- ProcessPayment()

Verification

Attributes:

- Verification ID
- Status
- Inspection Date

Behaviors:

- ApproveProperty()
- RejectProperty()

3.5 High-Level Use Case Diagram for the Proposed System

The Use Case Diagram represents interactions between users and the ShortStay system.

The main actors are:

- Guest
- Host
- Admin
- Accountant
- Verifier

Main use cases:

Guest

- Register/Login
- Search Properties
- View Property Details
- Book Accommodation
- Make Payment
- Submit Review

Host

- Manage Profile
- Add Property
- Update Property Details
- Manage Bookings

Admin

- Manage Users
- Manage Properties
- Monitor System

Accountant

- Monitor Transactions
- Track Revenue & Commission
- Process Refunds

Verifier

- View Inspection Queue
- Submit Inspection Report
- Approve/Reject Verification

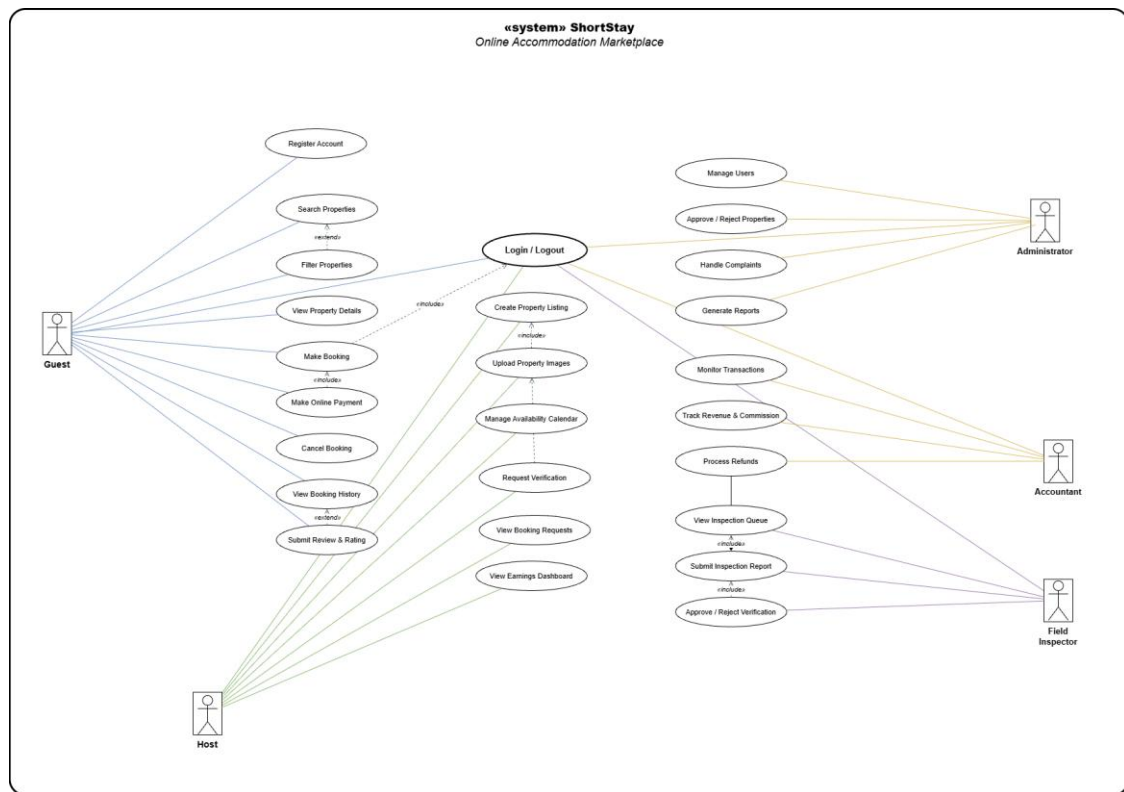


Figure 1 - Use Case Diagram

3.6 ER Diagram for the Proposed System

The Entity Relationship Diagram represents the database structure of the ShortStay system.

Main entities:

- User
- Host
- Guest
- Staff
- Property
- Booking
- Payment
- Review
- Inspection
- Notification
- Complain

Main relationships:

- A User can create multiple Properties.
- A Guest can make multiple Bookings.
- A Property can have multiple Bookings.
- A Booking has one Payment.
- A Property has one Inspection record.
- A Guest can submit Reviews.

- Property
- Booking
- Payment
- Complain
- Payout
- Inspection

The diagram shows:

- Classes
- Attributes
- Methods
- Relationships

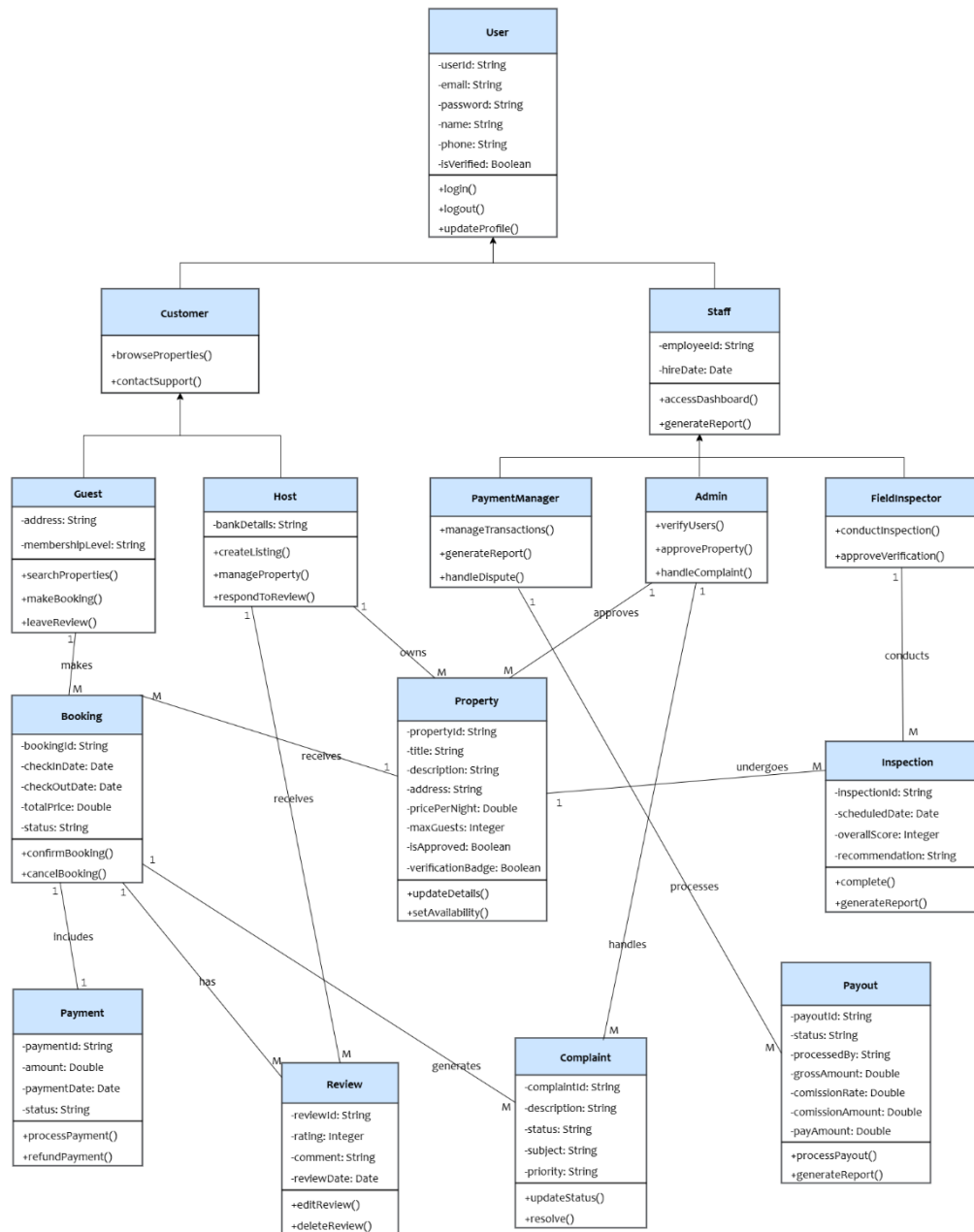


Figure 3 - Class Diagram

3.8 User Interface Designing for the System

The user interface of ShortStay was designed to provide a simple and efficient experience for all users.

The UI design focuses on:

- Simple navigation
- Clear information display
- Responsive layouts
- Role-based dashboards

Major interfaces include:

Guest Interface

- Login page

The screenshot displays a web browser window with the URL 'shortstay.com/login'. The page features a header with the 'Shortstay' logo, navigation links for 'HOME', 'CONTACT', and 'ABOUT', and a 'Login' button. The main content area is a 'Sign Up Free' form. The form includes input fields for 'First Name', 'Last Name', 'Email' (with the example 'example@gmail.com'), and 'Password'. Below these fields is a checkbox for 'Terms and condition' and a 'Create Account' button. Underneath the form, there is a section for 'Or sign up with:' featuring buttons for 'Google', 'Apple', and 'Twitter'. At the bottom of the form, a link says 'Already have an account?'. The browser's address bar and Windows taskbar are visible at the top and bottom of the window.

Figure 4 - User Interface for the Login Page

- Property search page.

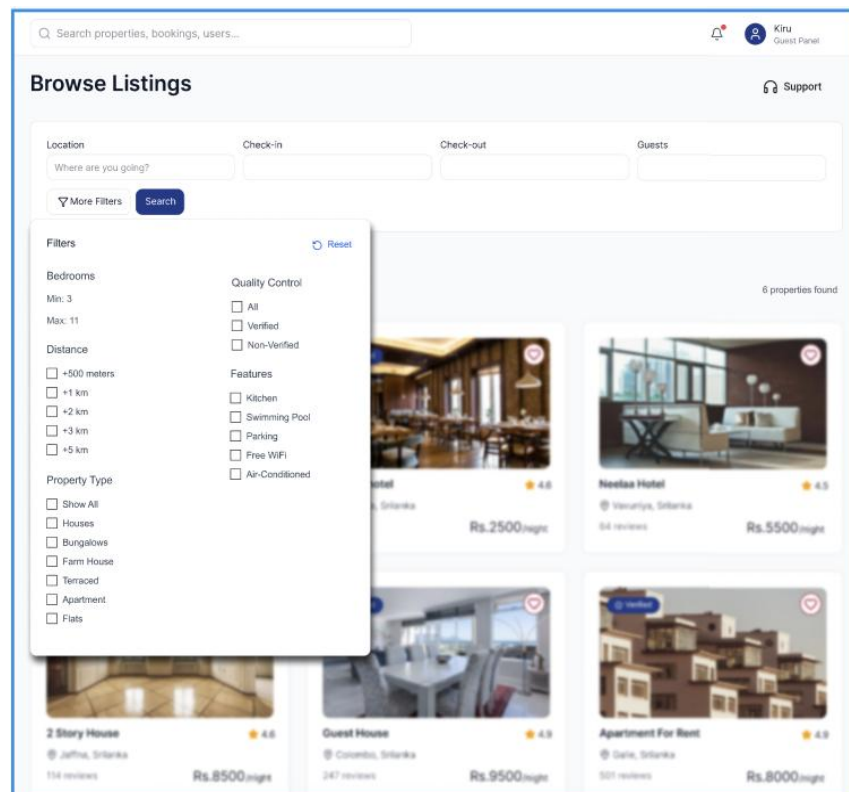


Figure 5 - User Interface for the Property search page

- Property details page

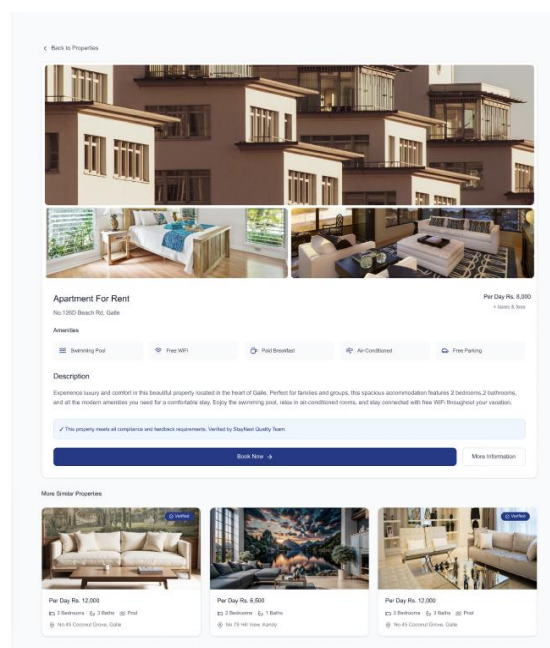


Figure 6 - User Interface for the Property details page

- Booking page

Back to Property

Billing Payment

Payment Type
☒ Credit Card

Credit Card Number
 1234 5678 9012 3456

Expiry Date (MM/YY)
 12/2028

CVV
 123

First Name
 John

Last Name
 Doe

Billing Address
 123 Main Street

City
 Colombo

Province
 Western

Postal Code
 00100

Mobile Number
 +94 77 123 4567

Email Address
 john.doe@email.com

Cost Breakdown

Number of Days
 3

Total cost of rent (3 days) Rs. 24,000

Service charges (20%) Rs. 1,200

Tax (10%) Rs. 2,400

Discount - Rs. 0

Total Charge Rs. 27,600

[Pay Now](#)

[Simulate Failed Payment](#)

Company
 About StayNest
 Careers
 Contact Us

Legal
 Member Terms
 Privacy Policy
 Terms & Conditions
 Cookies

Support
 support@shortstay.com
 0771234567

CHOOSE. STAY. CHILL
 Your trusted platform for short-stay rentals in Sri Lanka.

All property listings are subject to availability and verification. StayNest Ltd acts as an intermediary between property owners and guests. A commission fee applies to all bookings. Properties are covered under StayNest's full insurance policy. This site is governed under FCA regulations and complies with Sri Lanka property rental laws.
 © 2020 StayNest Ltd. All rights reserved.

Figure 7 - User Interface for the Booking page

Host Interface

- Host dashboard

shortstay

Search properties, bookings, users...

Ruban
Host Panel

Bookings
Manage reservations for your properties

Total Bookings
1

Confirmed
1

Pending
0

Completed
0

All Bookings

Booking ID	Guest	Property	Check-in	Check-out	Nights	Your Earning	Status	Actions
BK-2020-001	kiru kiru@email.com	Luxury Downtown Loft	2020-01-15	2020-01-20	5	25000 LKR Fee: 2500 LKR	confirmed	

My Listings

Bookings

Earnings

Reviews

Settings

Figure 8 - User Interface for the Host dashboard

- Property management page

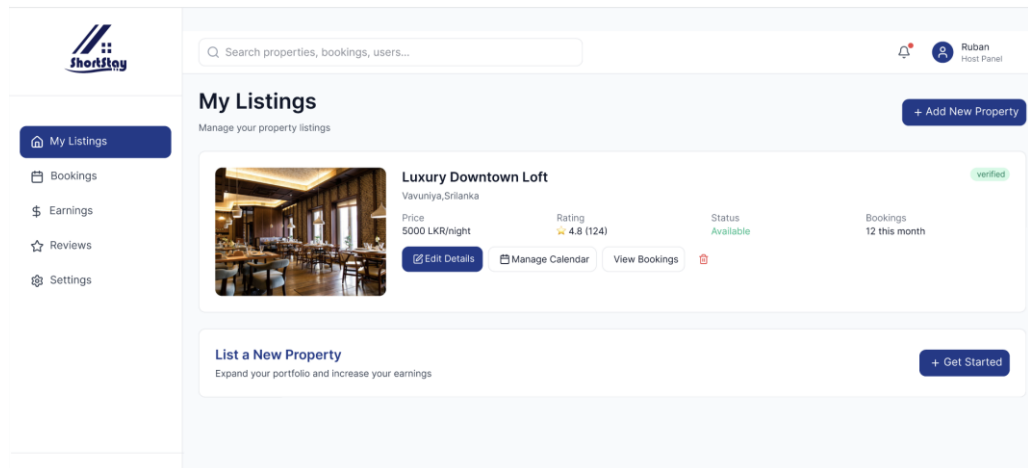


Figure 9 - User Interface for the Property management page

Admin Interface

- Admin dashboard

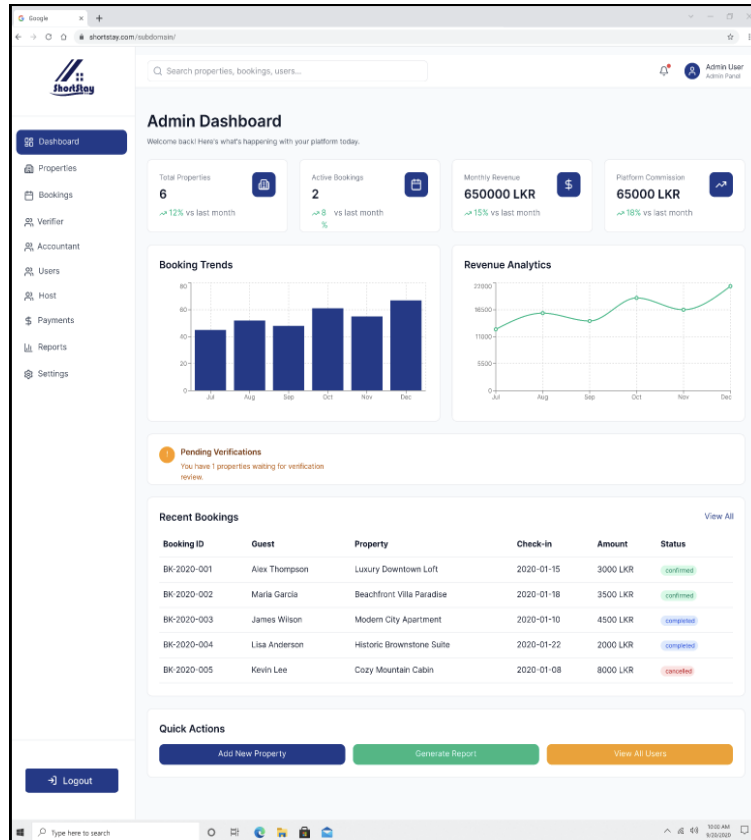
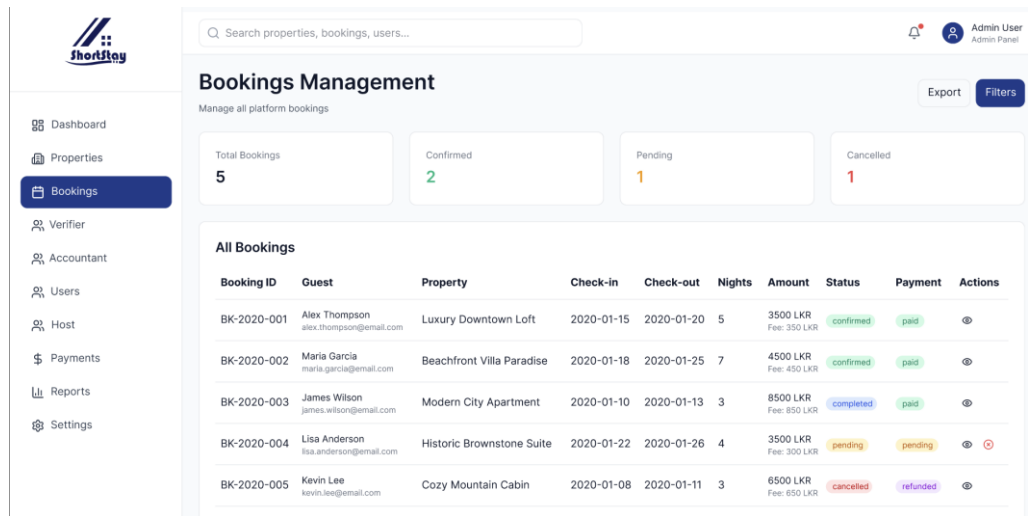


Figure 10 - User Interface for the Admin dashboard

- Booking management



Bookings Management
Manage all platform bookings

Search properties, bookings, users...

Export Filters

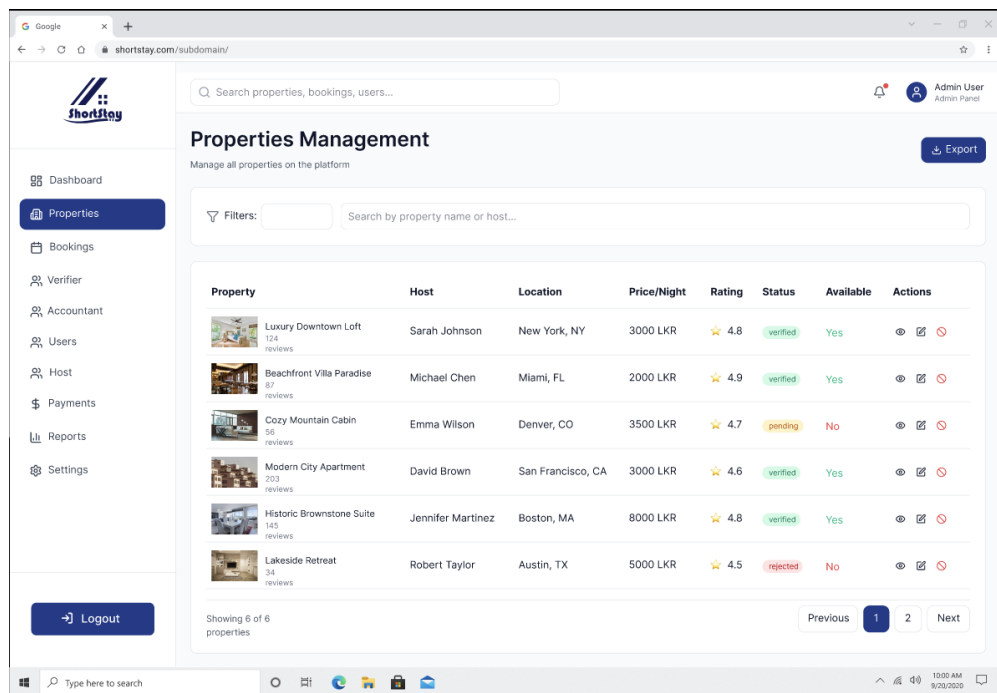
Total Bookings: 5
Confirmed: 2
Pending: 1
Cancelled: 1

All Bookings

Booking ID	Guest	Property	Check-in	Check-out	Nights	Amount	Status	Payment	Actions
BK-2020-001	Alex Thompson alex.thompson@email.com	Luxury Downtown Loft	2020-01-15	2020-01-20	5	3500 LKR Fee: 350 LKR	confirmed	paid	👁️
BK-2020-002	Maria Garcia maria.garcia@email.com	Beachfront Villa Paradise	2020-01-18	2020-01-25	7	4500 LKR Fee: 450 LKR	confirmed	paid	👁️
BK-2020-003	James Wilson james.wilson@email.com	Modern City Apartment	2020-01-10	2020-01-13	3	8500 LKR Fee: 850 LKR	completed	paid	👁️
BK-2020-004	Lisa Anderson lisa.anderson@email.com	Historic Brownstone Suite	2020-01-22	2020-01-26	4	3500 LKR Fee: 350 LKR	pending	pending	👁️ 🚫
BK-2020-005	Kevin Lee kevin.lee@email.com	Cozy Mountain Cabin	2020-01-08	2020-01-11	3	6500 LKR Fee: 650 LKR	cancelled	refunded	👁️

Figure 11 - User Interface for the Booking management

- Property management



Properties Management
Manage all properties on the platform

Search properties, bookings, users...

Export

Filters: Search by property name or host...

Property	Host	Location	Price/Night	Rating	Status	Available	Actions
Luxury Downtown Loft 124 reviews	Sarah Johnson	New York, NY	3000 LKR	★ 4.8	verified	Yes	👁️ 📄 🗑️
Beachfront Villa Paradise 97 reviews	Michael Chen	Miami, FL	2000 LKR	★ 4.9	verified	Yes	👁️ 📄 🗑️
Cozy Mountain Cabin 56 reviews	Emma Wilson	Denver, CO	3500 LKR	★ 4.7	pending	No	👁️ 📄 🗑️
Modern City Apartment 203 reviews	David Brown	San Francisco, CA	3000 LKR	★ 4.6	verified	Yes	👁️ 📄 🗑️
Historic Brownstone Suite 145 reviews	Jennifer Martinez	Boston, MA	8000 LKR	★ 4.8	verified	Yes	👁️ 📄 🗑️
Lakeside Retreat 34 reviews	Robert Taylor	Austin, TX	5000 LKR	★ 4.5	rejected	No	👁️ 📄 🗑️

Showing 6 of 6 properties

Previous 1 2 Next

Figure 12 - User Interface for the Property management

- Report Generation

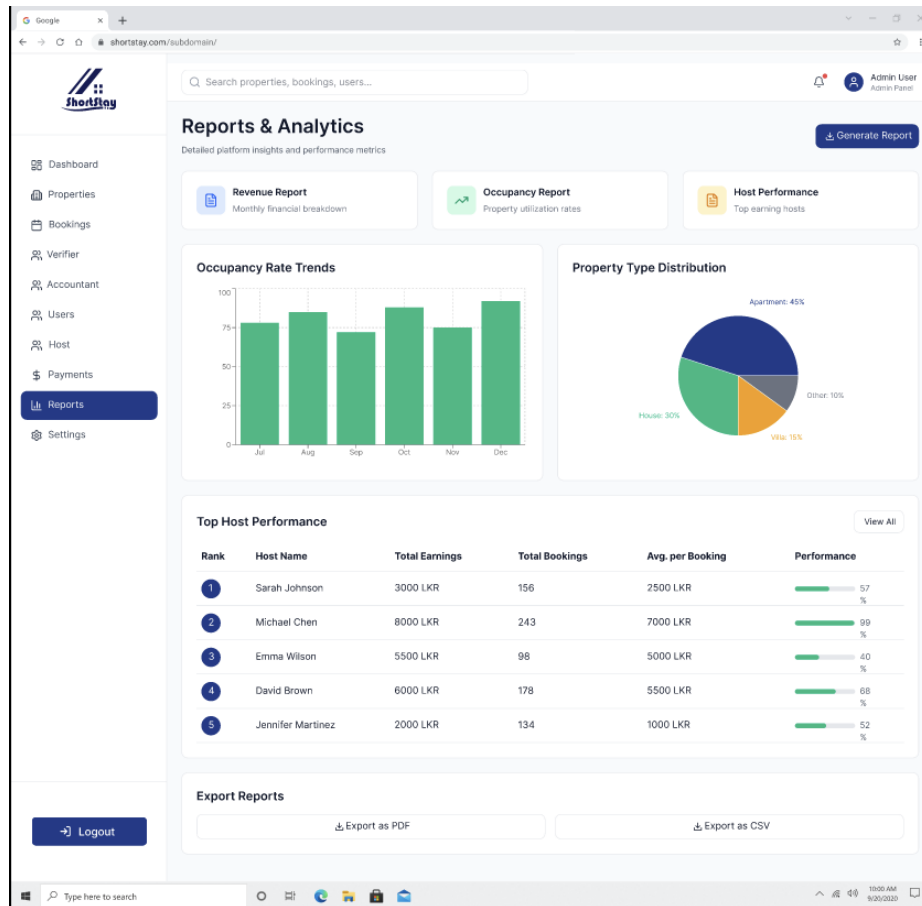


Figure 13 - User Interface for the Report Generation

Accountant Interface

- Payment dashboard

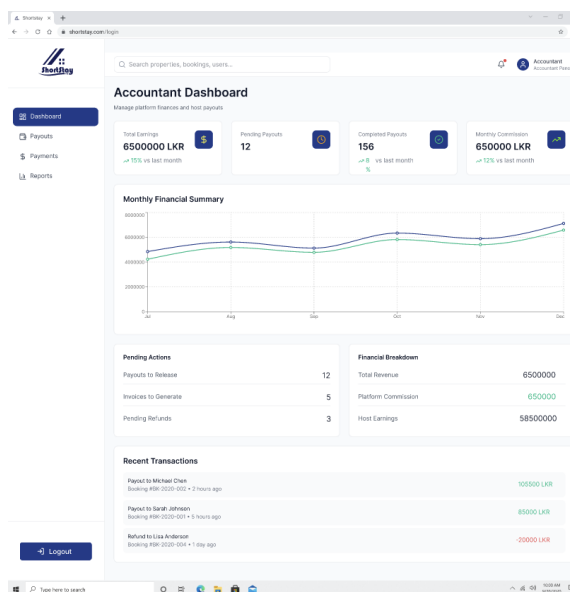


Figure 14 - User Interface for the Payment dashboard

- Transaction management

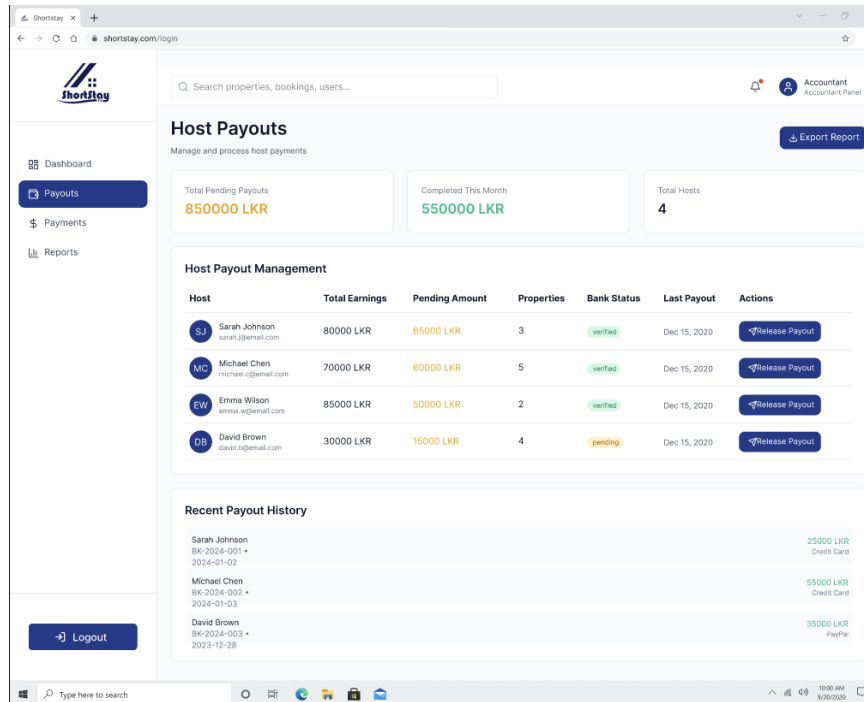


Figure 15 - User Interface for the Transaction management

Verifier Interface

- Verification dashboard

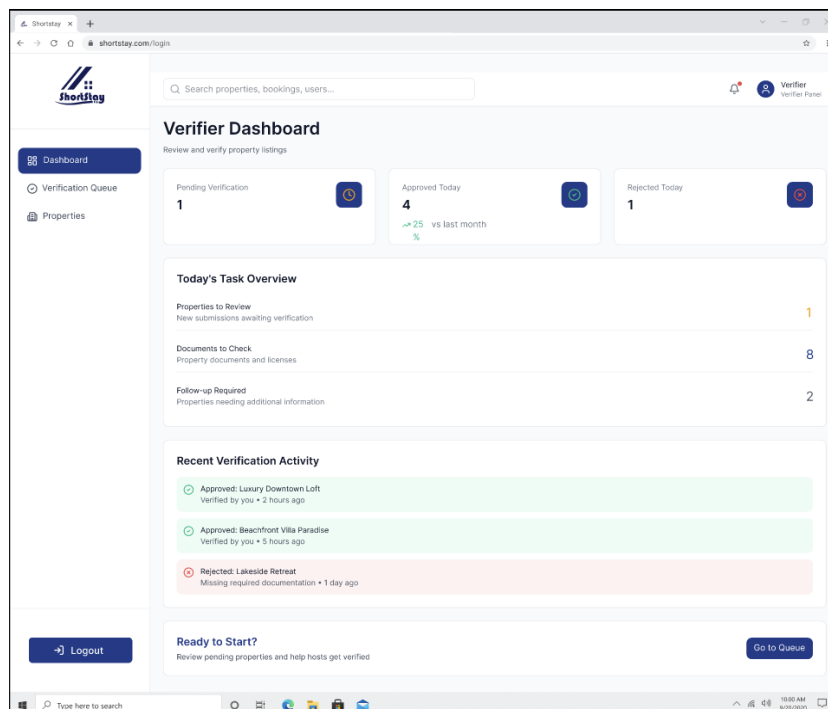


Figure 16 - User Interface for the Verification dashboard

- Property inspection interface

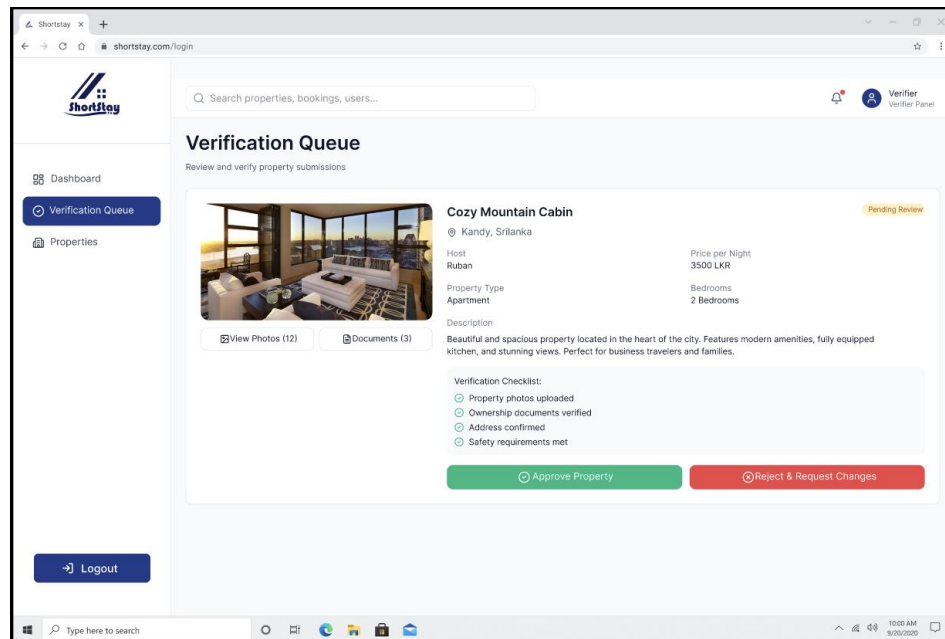


Figure 17 - User Interface for the Property inspection interface

The interface designs were created to ensure usability and consistency across the entire system.

Chapter 4 – IMPLEMENTATION

4.1 Introduction

This chapter is about how we built the ShortStay website. We made it after finishing the planning and design stages. The website lets guests find and book short-term stays. It also helps hosts, administrators, payment managers and field inspectors do their jobs. We used React for the websites part, Node.js and Express for the back part and MySQL to store data. We focused on making sure people can only do what they're supposed to do on the site. We also made it easy to list properties, manage bookings, handle payments online, verify properties and send reports and notifications.

The goal was to make a platform that's both secure and easy to use. Guests can book stays, while hosts and staff can manage their work. They can do it all through dashboards made just for their roles. The back part of the site is organized into parts. There are routes, controllers, models, middleware and utility files. This makes it easy to fix and add to the site later. The ShortStay website was built to be easy to use and maintain.

4.2 Implementation Environment

The implementation environment defines the hardware, software and development setup used to build and test the ShortStay system. Since the project is a web-based system, the main development environment required a modern web browser, Node.js runtime, MySQL database server and a code editor.

Table 1 - Development Environment Requirements

Category	Requirement / Tool	Purpose
Operating System	Windows 10/11, Linux or macOS	Used as the development platform for frontend and backend implementation
Processor	Intel Core i3 or above	Sufficient processing power for running local development servers
Memory	4 GB RAM minimum, 8 GB recommended	Needed to run the frontend, backend and database tools smoothly

Frontend Runtime	Node.js with npm	Required to install packages and run the React/Vite frontend
Backend Runtime	Node.js	Executes the Express REST API server
Database Server	MySQL Server	Stores users, properties, bookings, payments, reviews and inspection data
Database Tool	MySQL Workbench / SQL client	Used to create and manage the ShortStay database
Code Editor	Visual Studio Code	Used for writing and managing source code
API Testing Tool	Postman	Used for testing backend endpoints
Browser	Google Chrome / Microsoft Edge / Firefox	Used for frontend testing and user acceptance walkthroughs
Version Control	Git and GitHub	Used for source-code management and team collaboration

During development we run the frontend. Backend as separate services. The React frontend is served using the Vite development server. The Express backend runs as an API service. The backend talks to the MySQL database using Sequelize. We use environment variables to store configuration values. These values include database credentials, JWT secret keys, the URL for the frontend, the URL for the backend and keys for third-party APIs. The React frontend and backend services work together in this setup. The MySQL database and Sequelize help manage data. The environment variables keep our configuration safe and organized.

4.3 System Developed Tools and Technologies

The ShortStay system was implemented using modern web application technologies. The selected tools support modular development, fast interface building, secure authentication, role-based access control and relational data management.

Table 2 - System Development Tools and Technologies

Area	Technology	Usage in ShortStay
Frontend	React.js	Developed interactive pages and role-based dashboards

Frontend Routing	React Router DOM	Handled public, guest, host, admin, payment manager and inspector routes
Frontend API Access	Axios	Sent HTTP requests from the frontend to backend REST APIs
Frontend Styling	Tailwind CSS, CSS	Created responsive layouts and consistent user interface styles
Frontend Charts	Recharts	Displayed dashboard and report-related charts
Frontend Icons	Lucide React	Used icons for navigation and dashboard actions
Build Tool	Vite	Provided fast frontend development and production builds
Backend	Node.js and Express.js	Implemented REST API endpoints and backend business logic
Database ORM	Sequelize with mysql2	Mapped JavaScript models to MySQL database tables
Database	MySQL	Stored structured application data
Authentication	JWT and bcryptjs	Implemented secure login, password hashing and token-based sessions
Validation and Security	express-validator, express-rate-limit, CORS	Validated inputs, controlled access and reduced abuse attempts
File Uploading	Multer	Handled property, inspection and profile image uploads
Payments	Stripe	Supported secure online payment workflow
Email	Nodemailer / email templates	Sent verification, reset, booking and payment emails
Real-time Updates	Socket.IO	Supported real-time notification functionality
API Documentation	Swagger UI	Displayed backend API documentation for developers

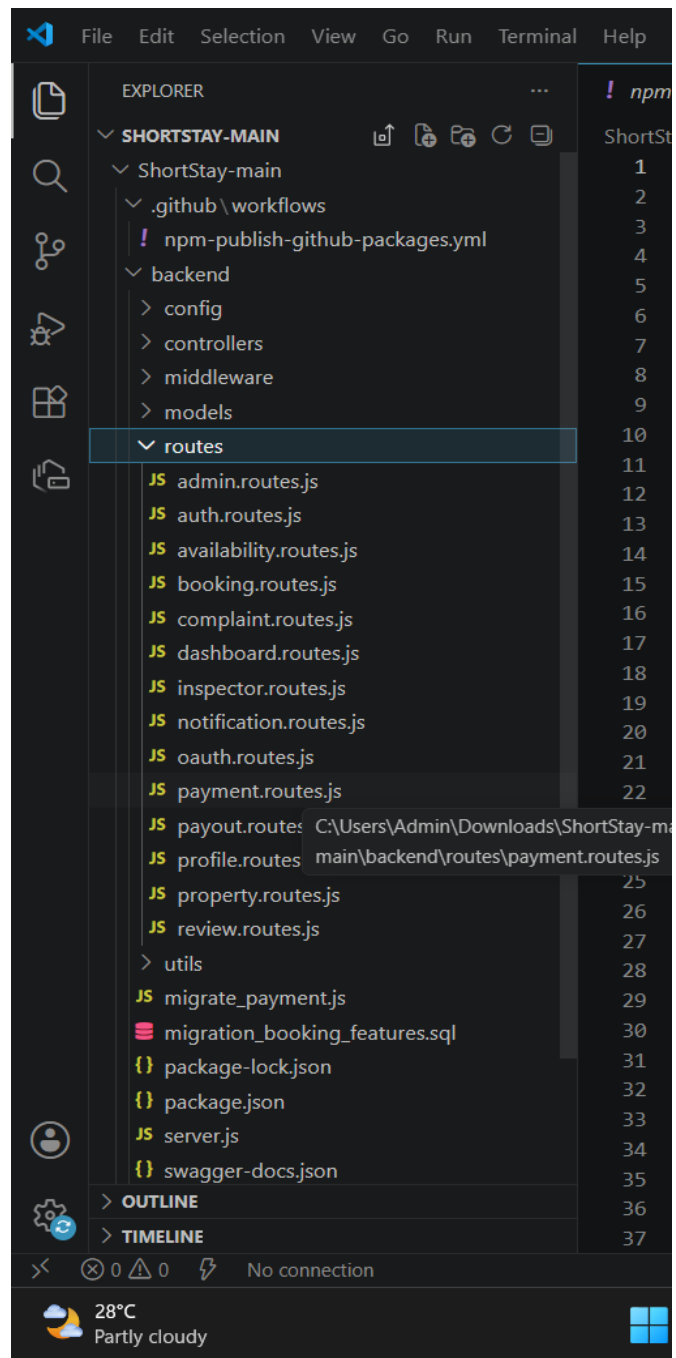


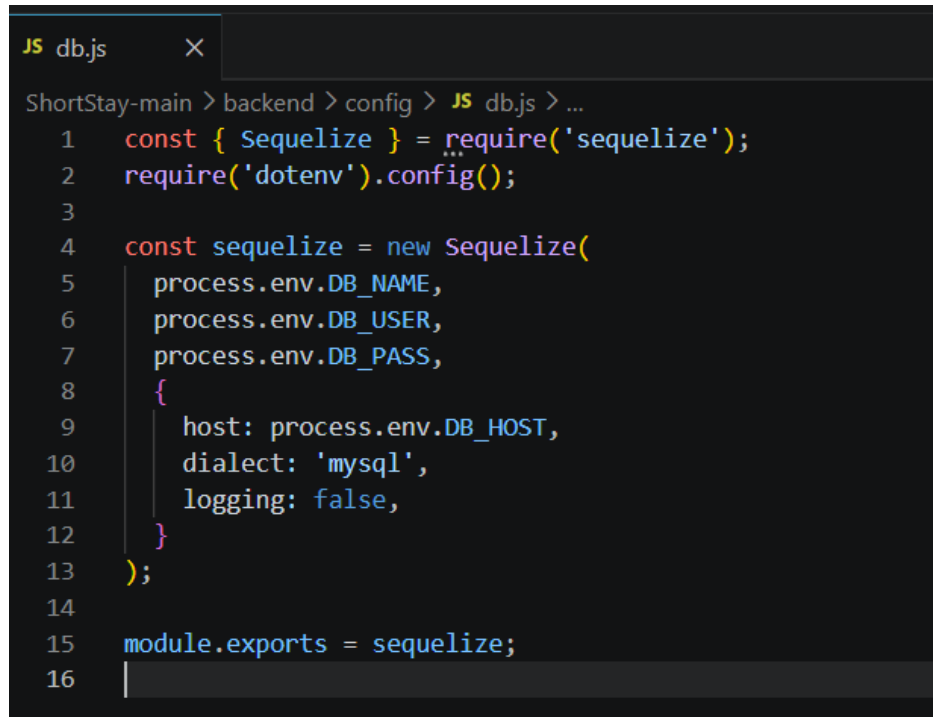
Figure 18 - System module folder structure of ShortStay

4.4 Major Code Segments

This section explains the most important code segments used in the system implementation. The code screenshots are selected from the actual project files because they show the main implementation logic of the ShortStay application.

4.4.1 Database Connection

The backend uses Sequelize to connect the Node.js application with the MySQL database. Database credentials are not hard-coded in the application. Instead, values such as database name, database user, password and host are loaded through environment variables. This approach improves security and allows the same project to be configured for different environments



```
JS db.js X
ShortStay-main > backend > config > JS db.js > ...
1  const { Sequelize } = require('sequelize');
2  require('dotenv').config();
3
4  const sequelize = new Sequelize(
5    process.env.DB_NAME,
6    process.env.DB_USER,
7    process.env.DB_PASS,
8    {
9      host: process.env.DB_HOST,
10     dialect: 'mysql',
11     logging: false,
12   }
13 );
14
15 module.exports = sequelize;
16
```

Figure 19 - Code segment for database connection

4.4.2 Server Initialization and API Route Registration

The Express server is the main entry point of the backend. It configures middleware such as request logging, response-time monitoring, rate limiting, CORS, JSON parsing and Passport initialization. After middleware configuration, each system module is registered as a separate API route. This keeps the backend modular and makes the system easier to maintain.

```

JS db.js JS server.js X
ShortStay-main > backend > JS server.js > ...
21
22 // — Non-functional middleware —
23 app.use(requestLogger); // HTTP request log (morgan)
24 app.use(responseTimeMonitor); // X-Response-Time header + slow-req warning
25 app.use(globalRateLimit); // 100 req / 15 min / IP
26
27 app.use(cors({
28   origin: process.env.FRONTEND_URL,
29   credentials: true
30 }));
31
32 // — Stripe webhook — must receive RAW body before express.json() parses it —
33 app.use(
34   '/api/payments/webhook',
35   express.raw({ type: 'application/json' })
36 );
37
38 app.use(express.json());
39 app.use(passport.initialize());
40 app.use('/uploads', express.static('uploads'));
41 app.use('/uploads/profiles', express.static('uploads/profiles'));
42
43 // — API Routes —
44 app.use('/api/auth', authRateLimit, require('./routes/auth.routes'));
45 app.use('/api/auth', authRateLimit, require('./routes/oauth.routes'));
46 app.use('/api/properties', require('./routes/property.routes'));
47 app.use('/api/bookings', require('./routes/booking.routes'));
48 app.use('/api/payments', require('./routes/payment.routes'));
49 app.use('/api/payouts', require('./routes/payout.routes'));
50 app.use('/api/reviews', require('./routes/review.routes'));
51 app.use('/api/complaints', require('./routes/complaint.routes'));
52 app.use('/api/admin', require('./routes/admin.routes'));
53 app.use('/api/inspector', require('./routes/inspector.routes'));
54 app.use('/api/availability', require('./routes/availability.routes'));
55 app.use('/api/profile', require('./routes/profile.routes'));
56 app.use('/api/notifications', require('./routes/notification.routes'));
57 app.use('/api/dashboard', require('./routes/dashboard.routes'));

```

Figure 20 - Code segment for Express server and route registration

4.4.3 Login, Account Checks and JWT Generation

The authentication controller handles user login. It checks whether the user exists, whether the account is suspended, whether email verification is required, and whether the entered password matches the hashed password stored in the database. The system also includes a brute-force protection mechanism that locks the account temporarily after repeated failed login attempts. When the login is successful, a JWT token is generated and returned to the frontend.

```

ShortStay-main > backend > controllers > JS auth.controller.js > ...
224 exports.login = async (req, res) => {
225   try {
226     const { email, password, remember_me } = req.body;
227
228     const user = await User.findOne({ where: { email } });
229     if (!user) {
230       return res.status(404).json({ message: 'User not found' });
231     }
232
233     // — DB-persisted brute-force check —
234     if (user.locked_until && new Date() < new Date(user.locked_until)) {
235       const remainingTime = Math.ceil(
236         (new Date(user.locked_until) - Date.now()) / 60000
237       );
238       return res.status(429).json({
239         message: `Too many failed login attempts. Try again in ${remainingTime} minutes
240       });
241     }
242     // Auto-reset when lock period has passed
243     if (user.locked_until && new Date() >= new Date(user.locked_until)) {
244       await user.update({ failed_login_count: 0, last_failed_login: null, locked_until:
245     }
246
247     // Check if suspended
248     if (user.is_suspended) {
249       return res.status(403).json({
250         message: 'Your account has been suspended.',
251         reason: user.suspended_reason || 'Please contact support.'
252       });
253     }
254
255     // Block Google-only accounts (no password set) from password login
256     if (!user.password) {
257       return res.status(400).json({
258         message: 'This account uses Google login. Please sign in with Google.',
259       });
260     }

```

Figure 21 - Code segment for login and JWT generation

4.4.4 JWT Authentication Middleware

Protected backend routes use JWT middleware to ensure that only authenticated users can access restricted API endpoints. The middleware checks for a Bearer token in the request header, verifies the token using the JWT secret and loads the related user record. It also checks whether the account is suspended or whether the token has been invalidated after logout or password change.

```

ShortStay-main > backend > middleware > JS auth.middleware.js > [2] authMiddleware
1  const jwt = require('jsonwebtoken');
2  require('dotenv').config();
3
4  const authMiddleware = async (req, res, next) => {
5    const authHeader = req.headers.authorization;
6
7    if (!authHeader || !authHeader.startsWith('Bearer ')) {
8      return res.status(401).json({ message: 'No token provided' });
9    }
10
11    const token = authHeader.split(' ')[1];
12
13    try {
14      const decoded = jwt.verify(token, process.env.JWT_SECRET);
15
16      const { User } = require('../models/index');
17      const user = await User.findBypk(decoded.user_id);
18
19      if (!user) {
20        return res.status(404).json({ message: 'User not found' });
21      }
22
23      // Check if token was issued before the invalidation timestamp
24      // (covers logout, password change, and any admin-forced invalidation)
25      if (user.tokens_valid_after) {
26        const tokenIssuedAt = new Date(decoded.iat * 1000);
27        if (tokenIssuedAt < new Date(user.tokens_valid_after)) {
28          return res.status(401).json({
29            message: 'Token has been invalidated. Please login again.',
30          });
31        }
32      }
33
34      if (user.is_suspended) {
35        return res.status(403).json({
36          message: 'Your account has been suspended.',
37          reason: user.suspended_reason || 'Please contact support.',

```

Figure 22 - Code segment for JWT authentication middleware

4.4.5 Frontend Role-Based Route Protection

The frontend also contains role-based route protection. The ProtectedRoute component checks the currently logged-in user and allows access only to pages that match the user role. For example, a guest can access guest pages, while the admin dashboard is restricted to the admin role. This improves the user experience and prevents unauthorized users from viewing role-specific screens.

```

90      /* Guest */
91      <Route path="/guest" element={<ProtectedRoute role="guest"><GuestDashboard /></ProtectedRoute>} />
92      <Route path="/guest/browse" element={<ProtectedRoute role="guest"><BrowseListings /></ProtectedRoute>} />
93      <Route path="/guest/property/:id" element={<ProtectedRoute role="guest"><GuestPropertyDetail /></ProtectedRoute>} />
94      <Route path="/guest/bookings" element={<ProtectedRoute role="guest"><MyBookings /></ProtectedRoute>} />
95      <Route path="/guest/pay/:bookingId" element={<ProtectedRoute role="guest"><PaymentPage /></ProtectedRoute>} />
96      <Route path="/guest/wallet" element={<ProtectedRoute role="guest"><GuestWallet /></ProtectedRoute>} />
97      <Route path="/guest/reviews" element={<ProtectedRoute role="guest"><GuestReviews /></ProtectedRoute>} />
98      <Route path="/guest/settings" element={<ProtectedRoute role="guest"><GuestSettings /></ProtectedRoute>} />
99
100     /* Host */
101     <Route path="/host" element={<ProtectedRoute role="host"><HostDashboard /></ProtectedRoute>} />
102     <Route path="/host/listings" element={<ProtectedRoute role="host"><HostListings /></ProtectedRoute>} />
103     <Route path="/host/listings/new" element={<ProtectedRoute role="host"><PropertyForm /></ProtectedRoute>} />
104     <Route path="/host/listings/edit/:id" element={<ProtectedRoute role="host"><PropertyForm /></ProtectedRoute>} />
105     <Route path="/host/bookings" element={<ProtectedRoute role="host"><HostBookings /></ProtectedRoute>} />
106     <Route path="/host/earnings" element={<ProtectedRoute role="host"><HostDashboard /></ProtectedRoute>} />
107     <Route path="/host/reviews" element={<ProtectedRoute role="host"><HostReviews /></ProtectedRoute>} />
108     <Route path="/host/settings" element={<ProtectedRoute role="host"><GuestSettings /></ProtectedRoute>} />
109
110     /* Admin */
111     <Route path="/admin/dashboard" element={<ProtectedRoute role="admin"><AdminDashboard /></ProtectedRoute>} />
112     <Route path="/admin/users" element={<ProtectedRoute role="admin"><AdminUsers /></ProtectedRoute>} />
113     <Route path="/admin/properties" element={<ProtectedRoute role="admin"><AdminProperties /></ProtectedRoute>} />
114     <Route path="/admin/bookings" element={<ProtectedRoute role="admin"><AdminPayments /></ProtectedRoute>} />
115     <Route path="/admin/payments" element={<ProtectedRoute role="admin"><AdminPayments /></ProtectedRoute>} />
116     <Route path="/admin/reports" element={<ProtectedRoute role="admin"><AdminReports /></ProtectedRoute>} />
117     <Route path="/admin/verifier" element={<ProtectedRoute role="admin"><AdminProperties /></ProtectedRoute>} />
118     <Route path="/admin/accountant" element={<ProtectedRoute role="admin"><AdminPayments /></ProtectedRoute>} />
119     <Route path="/admin/host" element={<ProtectedRoute role="admin"><AdminUsers /></ProtectedRoute>} />
120     <Route path="/admin/settings" element={<ProtectedRoute role="admin"><AdminDashboard /></ProtectedRoute>} />
121
122     /* Inspector */
123     <Route path="/inspector/inspections" element={<ProtectedRoute role="field_inspector"><InspectorDashboard /></ProtectedRoute>} />
124     <Route path="/inspector/settings" element={<ProtectedRoute role="field_inspector"><InspectorDashboard /></ProtectedRoute>} />

```

Figure 23 - Code segment for frontend protected routes

4.4.6 Frontend API Communication

The frontend uses a reusable Axios instance to communicate with the backend REST API. The base URL is built dynamically from the current hostname, allowing the same build to work on localhost and over the local network (useful for mobile device testing). A request interceptor attaches the JWT token from local storage to every outgoing request. A response interceptor detects 401 Unauthorized responses, clears the stored session, and redirects the user to the login page.

```

ShortStay-main > ShortStay-main > frontend > src > JS api.js > ...
1  import axios from 'axios';
2  import { mockRequest } from './mockApi';
3
4  const MOCK = import.meta.env.VITE_MOCK_MODE === 'true';
5
6  const API_PORT = 5000;
7  const baseURL = window.location.hostname === 'localhost'
8    ? `http://localhost:${API_PORT}/api`
9    : `http://${window.location.hostname}:${API_PORT}/api`;
10
11  const api = axios.create({ baseURL });
12
13  api.interceptors.request.use(cfg => {
14    const token = localStorage.getItem('token');
15    if (token) cfg.headers.Authorization = `Bearer ${token}`;
16    return cfg;
17  });

```

Figure 24 - Code segment for Axios API helper

4.4.7 Booking Creation and Availability Validation

The booking controller validates property availability before creating a booking. It checks whether the selected property is approved, validates check-in and checkout dates, calculates the number of nights and checks whether all required dates are available. This logic prevents invalid bookings and reduces double-booking issues. The controller also enforces platform-level booking rules retrieved from system settings, such as a minimum number of nights and a maximum advance-booking period, and rejects check-in dates set in the past.

```
ShortStay-main > ShortStay-main > backend > controllers > JS booking.controller.js > makeBooking > makeBooking
69 exports.makeBooking = async (req, res) => {
70   try {
71     const { property_id, checkin_date, checkout_date } = req.body;
72
73     // 1. Validate property
74     const property = await Property.findOne({
75       where: { property_id, is_approved: true },
76     });
77     if (!property) {
78       return res.status(404).json({ message: 'Property not found or not approved' });
79     }
80
81     // 2. Validate dates
82     const checkin = new Date(checkin_date);
83     const checkout = new Date(checkout_date);
84     const nights = Math.ceil((checkout - checkin) / (1000 * 60 * 60 * 24));
85
86     // Fetch global settings
87     const settings = await getPlatformSettings();
88
89     if (nights <= 0) {
90       return res.status(400).json({ message: 'Invalid dates: checkout must be after checkin' });
91     }
92     const today = new Date();
93     today.setHours(0, 0, 0, 0);
94     if (checkin < today) {
95       return res.status(400).json({ message: 'Check-in date cannot be in the past' });
96     }
97     if (nights < settings.minBookingDays) {
98       return res.status(400).json({ message: `Minimum booking duration is ${settings.minBookingDays} nights` });
99     }
100
101     const daysInAdvance = Math.ceil((checkin - new Date()) / (1000 * 60 * 60 * 24));
102     if (daysInAdvance > settings.maxAdvanceBooking) {
103       return res.status(400).json({ message: `Cannot book more than ${settings.maxAdvanceBooking} days in advance` });
104     }
105   }
106 }
```

Figure 25 - Code segment for booking creation and availability validation

4.4.8 Payment Intent Creation

The payment module supports online payment processing. The Stripe PaymentIntent flow Before creating the Stripe PaymentIntent, the system checks that the booking belongs to the logged-in guest and that its status is approved (i.e., the host has already approved the booking). It also checks whether a completed payment already exists for that booking to prevent duplicate charges. A membership-based discount (silver/gold/platinum) is applied to the final amount before the Stripe PaymentIntent is created.

```

ShortStay-main > ShortStay-main > backend > controllers > JS payment.controller.js > ...
This document contains many non-basic ASCII unicode characters  Disable Non ASCII Highlight
97 exports.createPaymentIntent = async (req, res) => {
98   try {
99     const stripe = getStripe();
100    const { booking_id } = req.body;
101
102    const booking = await Booking.findByPk(booking_id, {
103      include: [{ model: Property, as: 'property', attributes: ['title', 'address', 'host_id'] }],
104    });
105    if (!booking) return res.status(404).json({ message: 'Booking not found' });
106    if (booking.guest_id !== req.user.user_id)
107      return res.status(403).json({ message: 'Not authorized' });
108    if (booking.status !== 'approved')
109      return res.status(400).json({ message: 'Payment can only be made for approved bookings' });
110
111    const existingPayment = await Payment.findOne({ where: { booking_id, payment_status: 'completed' } });
112    if (existingPayment) {
113      return res.status(400).json({ message: 'Payment already processed' });
114    }
115
116    const guestUser = await User.findByPk(booking.guest_id);
117    let discountPct = 0;
118    if (guestUser?.membership_level === 'silver') discountPct = 0.01;
119    else if (guestUser?.membership_level === 'gold') discountPct = 0.02;
120    else if (guestUser?.membership_level === 'platinum') discountPct = 0.03;
121
122    const discountAmount = booking.total_price * discountPct;
123    const finalAmount = booking.total_price - discountAmount;
124
125    // Convert to cents for stripe
126    const amountCents = Math.round(finalAmount * 100);
127
128    const paymentIntent = await stripe.paymentIntents.create({
129      amount: amountCents,
130      currency: 'usd',

```

Figure 26 - Code segment for Stripe Payment Intent creation

4.5 System module structure

The ShortStay project is divided into frontend, backend and database layers. This separation makes the application easier to understand, test and extend. The frontend manages the user interface and user interaction, the backend handles business logic and API endpoints, and the database stores persistent records.

Table 3 - System Module Structure

Layer / Module	Main Files or Folders	Description
Frontend public pages	frontend/src/pages/public	Home, About, Contact, property detail and access portal pages
Frontend authentication pages	frontend/src/pages/auth	Login, registration and forgot-password interfaces
Guest module	frontend/src/pages/guest	Browse listings, view property details, make payments, view bookings, wallet, reviews and settings
Host module	frontend/src/pages/host	Host dashboard, listing management, property form, bookings and reviews

Admin module	frontend/src/pages/admin	Admin dashboard, user management, property approval, complaints, payments and reports
Payment manager module	frontend/src/pages/payment_manager	Payment dashboard and payment reports
Field inspector module	frontend/src/pages/inspector	Inspection dashboard and property verification workflow
Reusable UI components	frontend/src/components	Dashboard layout, sidebar, top bar, public navigation, footer and role switcher
Backend configuration	backend/config	Database, email, passport and Swagger configuration
Backend controllers	backend/controllers	Business logic for auth, booking, property, payment, reports, inspection and other modules
Backend routes	backend/routes	REST API route definitions for each module
Backend middleware	backend/middleware	Authentication, authorization, validators, upload handlers and system middleware
Backend models	backend/models	Sequelize models and relationships for database tables
Backend utilities	backend/utills	Activity logger, notification sender, email templates, membership logic, cron job and websocket utilities
Database script	Files/ShortstayNew.sql	SQL structure for user, property, booking, payment, review, inspection and related tables

The backend follows a controller-based structure. Each route file forwards requests to the relevant controller, and controllers use Sequelize models to read from or write to the database. Middleware files are used for repeated cross-cutting tasks such as authentication, authorization, file uploading and input validation.

4.6 Re-usable Components

Reusable components and utilities were used to reduce code duplication and improve consistency across the system. These reusable components support maintainability because a common layout, navigation bar, sidebar, authentication state and API handler can be changed in one place instead of updating every page separately.

Table 4 - Reusable Components and Utilities

Reusable Component / Utility	Location	Reusability Purpose
DashboardLayout	frontend/src/components/DashboardLayout.jsx	Provides a consistent dashboard layout for role-based panels
Sidebar	frontend/src/components/Sidebar.jsx	Displays role-specific side navigation links
TopBar	frontend/src/components/TopBar.jsx	Shows the page header, search field and user information
PublicNav	frontend/src/components/PublicNav.jsx	Common navigation bar for public pages
Footer	frontend/src/components/Footer.jsx	Common footer for the public and dashboard pages
AuthContext	frontend/src/AuthContext.jsx	Stores login state and user role across the frontend
API helper	frontend/src/api.js	Central place for backend API calls and token attachment
Auth middleware	backend/middleware/auth.middleware.js	Reusable JWT validation for protected backend APIs
Role middleware	backend/middleware/role.middleware.js	Reusable role permission checks for backend APIs
Validation middleware	backend/middleware/validators.js and validate.middleware.js	Reusable input validation rules
Upload middleware	backend/middleware/upload*.js	Reusable file upload handling for properties, profiles and inspections
Activity logger	backend/utls/activityLogger.js	Records important user/system activities
Email templates	backend/utls/emailTemplates.js	Reusable email layouts for verification, booking and payment workflows
Notification utility	backend/utls/notify.js	Reusable in-app notification creation
Booking expiry cron	backend/utls/bookingExpiryCron.js	Automatically handles expired pending bookings
Websocket utility	backend/utls/websocket.js	Reusable real-time notification setup

Using reusable components helped the project maintain a consistent user interface across all roles. It also improved development speed because common functions such as authentication, authorization, notifications and API handling did not need to be rewritten for every module.

Chapter 5 – EVALUATION

5.1 Introduction

This chapter evaluates the ShortStay system by describing the testing approach, testing techniques, test plan, test cases and user acceptance testing. Evaluation is important because it confirms whether the developed system satisfies the functional and non-functional requirements identified in the earlier chapters. The system was evaluated across major modules such as authentication, property management, booking management, payment management, role-based access, reports, reviews, complaints and property inspection.

The main objective of system evaluation was to ensure that the application works correctly for each user role and that data is processed accurately between the frontend, backend and database. Testing also focused on security-related areas such as protected routes, JWT validation, user role restrictions, password hashing, account suspension and rate limiting.

5.2 System Testing

System testing was carried out to validate the overall behaviour of the ShortStay application. The frontend was checked using the browser, while backend API endpoints were tested using Postman and direct route verification. Database-related functions were verified by checking whether records such as users, properties, bookings and payments were created, updated and retrieved correctly.

The system was evaluated module by module and then as a complete end-to-end flow. For example, a host can create a property listing, the property can be approved, a guest can search for the property, make a booking, complete the payment, and later provide a review. This end-to-end flow confirms the interaction between multiple modules.

5.3 Testing Techniques

The following testing techniques were used or prepared for evaluating the ShortStay application.

Table 5 - Testing Techniques

Testing Technique	Description	Application in ShortStay
-------------------	-------------	--------------------------

Black-box testing	Tests system behaviour without checking internal code	Used for login, registration, search, booking, payment and dashboard workflows
White-box testing	Tests internal logic and code paths	Used for authentication logic, route protection, booking validation and payment processing code
Input validation testing	Checks whether invalid inputs are rejected properly	Used for email, password, dates, property details, payment and review forms
Security testing	Checks authentication and access control	Used for JWT token validation, role-based access, account suspension and invalid token handling
Usability testing	Checks ease of use and interface clarity	Used for public pages, guest dashboard, host dashboard and admin screens
Regression testing	Retests existing features after changes	Used after bug fixing or module updates to confirm previous functions still work

5.4 Testing Methodologies

Testing was organized using different testing levels. This helped identify errors at the smallest module level and also at the full system level.

Table 6 - Testing Methodologies

Testing Methodology	Purpose	Example in ShortStay
Unit testing	Tests individual functions or modules separately	Testing password hashing, JWT generation, date range generation and refund calculation logic
Integration testing	Tests whether connected modules work together	Testing frontend login request with backend authentication and database user records
System testing	Tests the full system as one application	Testing complete guest booking and payment workflow
User acceptance testing	Checks whether the system meets user expectations	Guest, host and admin users perform realistic tasks and provide feedback
Performance observation	Checks response and usability under normal use	Checking page loading, API response behaviour and dashboard data loading

Compatibility testing	Checks the system in different browsers	Testing screens in Chrome, Edge or Firefox
-----------------------	---	--

5.5 System Test Plan

The system test plan defines what should be tested, which tools should be used and what result is expected. The test plan covers the major modules that are important for the successful operation of the ShortStay system.

Table 7 - System Test Plan

Test Area	Modules Covered	Testing Method	Expected Outcome
Authentication and user management	Registration, login, logout, forgot password, profile, staff creation	Black-box, white-box, security testing	Only valid users can access the system and roles are assigned correctly
Role-based access control	Guest, host, admin, payment manager and field inspector routes	Security and integration testing	Users can access only the pages and APIs permitted for their role
Property management	Create, update, delete, upload images, request verification, approve/reject listing	Functional and integration testing	Property data is stored correctly and displayed to users
Search and filtering	BrowseListings and property detail pages	Black-box and usability testing	Guests can filter and view properties accurately
Booking management	Make booking, approve, reject, cancel, booking history	Functional, integration and boundary testing	Bookings are created only for valid available dates
Payment management	Stripe intent, manual payment, refund, receipt, payout and reports	Integration and security testing	Payments are linked to bookings and status is updated correctly
Inspection and verification	Inspector dashboard, submit inspection, approve badge	Functional and workflow testing	Property verification status is updated based on inspection results
Reviews and complaints	Create review, edit review, host response, complaint status	Functional testing	Users can submit feedback and admins can manage complaints

Reports and dashboards	Admin reports, payment manager reports, dashboards	System and UI testing	Dashboard totals and report results are displayed correctly
Non-functional requirements	Security, usability, maintainability and responsiveness	Usability, compatibility and performance observation	System remains user-friendly, secure and maintainable

5.6 System Test Cases

The following test cases were prepared to validate the main features of the ShortStay system. These cases can be used during final testing and project demonstration.

Table 8 - Test Cases for Authentication

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-LOG-001	Verify login with valid credentials	User should be redirected to the appropriate dashboard	User was successfully logged in and redirected to the Guest Dashboard	Pass
TC-LOG-002	Verify login with incorrect password	Login should fail with an Invalid Credentials message	Login failed and the invalid credentials message was displayed	Pass
TC-LOG-003	Verify login using an unverified email account	User should not be allowed to log in and should be prompted to verify the email	Login was blocked and the verification prompt was displayed.	Pass
TC-LOG-004	Verify logout functionality	User should be logged out and redirected to the login page	User session ended successfully and the login page was displayed	Pass
TC-REG-001	Verify user registration with valid details	User account should be created successfully and a verification email should be sent	User account was created successfully and the verification email was sent.	Pass
TC-REG-002	Verify registration using an existing email address	Registration should be rejected and an "Email already exists" message should be displayed.	Registration was rejected and the appropriate error message was displayed.	Pass

TC-REG-003	Verify registration with mandatory fields left blank	Validation messages should be displayed and the form should not be submitted.	Validation messages were displayed and registration was prevented.	Pass
TC-UP-001	Verify viewing user profile information	User profile information should be displayed correctly.	User profile information was displayed correctly.	Pass
TC-UP-002	Verify updating user profile details	Profile information should be updated successfully.	Profile information was updated successfully.	Pass
TC-UP-003	Verify uploading a profile picture	Profile picture should be uploaded and displayed.	Profile picture was uploaded and displayed successfully.	Pass

Table 9 - Test Cases for Property and Search Management

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-HST-001	Add a new property listing	Property should be created with Pending Approval status	Property was created successfully and marked as Pending Approval	Pass
TC-HST-002	Edit property information	Property information should be updated successfully	Property information was updated successfully	Pass
TC-HST-003	Delete property listing	Selected property should be removed	Property was deleted successfully	Pass
TC-HST-004	Upload property images	Images should be uploaded and displayed	Images were uploaded successfully and displayed correctly	Pass

TC-HST-005	Verify viewing booking requests	All pending booking requests should be displayed.	Booking requests were displayed correctly.	Pass
TC-HST-006	Verify accepting a booking request	Booking status should change to Approved and the guest should be notified.	Booking was approved successfully and the guest received the booking approval notification.	Pass
TC-SRH-001	Search property using keywords	Matching properties should be displayed	Matching properties were displayed successfully	Pass
TC-SRH-002	Filter properties by availability	Only available properties should be displayed	Available properties were displayed correctly	Pass
TC-SRH-003	Filter by property type	Only apartment listings should be displayed	Apartment listings were displayed correctly	Pass
TC-SRH-004	View property image gallery	Image gallery should open successfully	Image gallery opened successfully and images were displayed	Pass

Table 10 - Test Cases for Booking Management

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-BKG-001	Create booking for an available property	Booking should be created successfully	Booking was created successfully and confirmation was displayed	Pass
TC-BKG-002	Book unavailable dates	Booking should be rejected with an availability message	Booking was rejected and availability warning was displayed	Pass
TC-BKG-003	Booking with invalid date range	Validation error should be displayed	Validation error was displayed and booking was not created	Pass

	(Min. no of days restriction)			
TC-BKG-004	Cancel existing booking	Booking should be cancelled successfully	Booking was cancelled successfully and status was updated	Pass
TC-BKG-005	View booking history	Previous bookings should be displayed	Booking history was displayed correctly	Pass

Table 11 - Test Cases for Payment, Reports and Feedback

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-PAY-001	Successful payment	Payment should be processed and booking marked as Confirmed	Payment was completed successfully, booking status changed to Confirmed, and receipt was generated	Pass
TC-PAY-002	Payment with invalid card details	Payment should fail with an error message	Payment was declined and an appropriate error message was displayed	Pass
TC-PAY-003	View payment history	Completed payment records should be displayed	Payment history was displayed with correct transaction details	Pass
TC-PAY-004	Submit refund request	Refund request should be submitted successfully	Refund request was submitted and marked as Pending Review	Pass
TC-PAY-005	Cancel payment before confirmation	Payment should be cancelled without charging the user	Payment process was cancelled and no transaction was recorded	Pass

Table 12 - Test Cases for Inspection, Admin and Security

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-ADM-001	Verify administrator login with valid credentials	Administrator should be redirected to the Admin Dashboard.	Administrator logged in successfully and the dashboard was displayed.	Pass
TC-ADM-002	Verify viewing registered users	All registered users should be displayed with relevant details.	User list was displayed correctly with all registered users.	Pass
TC-ADM-003	Verify suspending a user account	Selected user account should be suspended successfully.	User account was suspended and the status changed to suspended.	Pass
TC-ADM-004	Verify approving a property listing	Property status should change to Approved and become visible to guests.	Property was approved successfully and became visible in the property listings.	Pass
TC-ADM-006	Verify rejecting a property listing	Property status should change to Rejected and the host should be notified.	Property was rejected successfully and notification was sent to the host.	Pass
TC-ADM-007	Verify generating system reports	System should generate and display the selected report.	Report was generated successfully.	Pass
TC-ADM-008	Verify viewing dashboard statistics	Dashboard should display user count, bookings, revenue, and pending verifications and approvals.	Dashboard statistics were displayed correctly.	Pass
TC-INS-001	Verify viewing pending property verification requests	Pending property verification requests should be displayed.	Pending verification requests were displayed successfully.	Pass
TC-INS-002	Verify approving a property after inspection	Property verification status should change to Verified.	Property verification status was updated to Verified.	Pass
TC-INS-003	Verify rejecting a property	Property should be rejected and the host notified.	Property was rejected and the host notification was sent successfully.	pass

TC-INS- 004	Verify viewing inspection history	Completed inspection records should be displayed.	Inspection history was displayed successfully.	Pass
TC-INS- 005	Verify submitting inspection remarks	Inspection remarks should be saved successfully.	Inspection remarks were saved successfully.	Pass
TC-SEC- 001	Verify access to protected pages without login	User should be redirected to the Login page.	User was redirected to the Login page successfully.	Pass
TC-SEC- 002	Verify SQL Injection protection during login	Login should fail and the request should be rejected.	SQL injection attempt was blocked successfully.	Pass
TC-SEC- 003	Verify Cross-Site Scripting (XSS) protection	Script should be sanitized and not executed.	Malicious script was sanitized and not executed.	Pass
TC-SEC- 004	Verify file upload validation	File upload should be rejected.	Invalid file upload was rejected successfully.	Pass
TC-SEC- 005	Verify password encryption	Password should be securely encrypted before storing.	Password was stored in encrypted form.	Pass

5.7 User acceptance test

User Acceptance Testing (UAT) verifies whether the completed system meets the expectations of the intended users. In the ShortStay system, UAT should involve representatives from each user role: guest, host, admin, payment manager and field inspector. Each user should be asked to perform realistic tasks and comment on ease of use, clarity, speed and completeness of the system.

Table 13 - User Acceptance Test Summary

User Role	UAT Task	Acceptance Criteria	Result
-----------	----------	---------------------	--------

Guest	Register, search property, view details, create booking and complete payment	Guest can complete the booking process without staff support	Accepted
Host	Create listing, update details, view bookings and check earnings	Host can manage listings and booking information clearly	Accepted
Admin	Manage users, approve properties, view complaints and reports	Admin can monitor and control system operations	Accepted
Payment Manager	View transactions, review payment status and generate payment report	Payment records can be tracked and summarized accurately	Accepted
Field Inspector	View assigned inspections and submit inspection result	Inspector can verify property information and submit recommendations	Accepted

The UAT results indicate that the major workflows of the ShortStay system are understandable and suitable for the selected user roles. Minor user interface improvements may still be completed before final deployment, especially for mobile responsiveness, clearer status labels and additional helper messages on important forms.

5.8 User Feedback

User feedback was considered important because ShortStay is used by different types of users with different technical knowledge. The main feedback areas are summarized below. These findings can be updated after collecting final questionnaire responses from real users.

Table 14 - User Feedback Summary

Feedback Area	Positive Feedback	Suggested Improvement
User interface	Users found the layout clean and easy to understand	Improve mobile view and make some dashboard cards more compact
Property search	Search and filtering options help guests find suitable accommodation	Add map-based search and more sorting options
Booking flow	Booking process is clear when dates are available	Add stronger visual calendar indicators for unavailable dates
Host dashboard	Hosts can quickly understand earnings and booking information	Add downloadable monthly statements

Admin functions	Admin has useful control over users, properties and reports	Add advanced audit log filtering
Payment management	Payment status tracking is useful for financial control	Add more local payment gateway support such as PayHere
Inspection process	Field inspector workflow improves trust in property quality	Add mobile-friendly inspection image upload and checklist view

5.9 Limitations of Evaluation

Although the system was evaluated through functional testing and usability observation, some limitations exist. The evaluation was performed within a controlled development environment and with a limited number of users. Large-scale deployment testing involving a higher number of simultaneous users was not conducted. Therefore, future evaluation can include stress testing and real-world user studies to further measure system scalability and reliability.

Chapter 6 – CONCLUSION

6.1 Introduction

This chapter concludes the ShortStay project by summarizing the implementation outcome, problems encountered during development, possible future improvements and lessons learnt. The ShortStay system was developed as a web-based marketplace for short-term accommodations. It provides a structured platform for guests to book accommodation and for hosts to publish and manage rental properties. The system also supports administrative control, payment management, inspections and reporting.

The project shows how a multi-role web application can be developed using React, Node.js, Express and MySQL. The final system includes the core modules required for an accommodation marketplace, such as user authentication, property listing management, booking management, payment processing, review handling, complaint handling and property verification.

6.2 Problems Encountered

Several technical and project-related problems were encountered during the development of the ShortStay system. These problems helped the team understand the complexity of building a real-world marketplace application.

- Handling multiple user roles was challenging because each role required separate dashboards, permissions and workflows. The system had to support guests, hosts, admins, payment managers and field inspectors without mixing their responsibilities.
- Designing secure authentication required additional care. Password hashing, JWT generation, token validation, account lockout and account suspension had to be implemented carefully to protect user accounts.
- Booking management was one of the most complex areas because the system needed to validate date ranges, calculate nights, check property availability and prevent double bookings.

- Payment handling required careful design because payment records must be linked correctly to bookings. Refunds, failed payments and payment reports also added complexity.
- Frontend and backend integration needed consistent API structures. Some frontend screens required data in a specific format, so API responses had to be designed clearly.
- File uploads for property, profile and inspection images required separate middleware and storage rules.
- Property verification required coordination between host, field inspector and admin workflows. This made the system more complex than a simple booking platform.
- Time management was difficult because the project included many modules and had to be completed within an academic schedule.
- Testing every role was time-consuming because each role has different screens and access permissions.

6.3 Future Improvements

Although the current ShortStay system includes the main marketplace features, several future improvements can be added to make the platform more complete, scalable and user-friendly.

- Develop a dedicated mobile application for Android and iOS users so that guests, hosts and inspectors can use the system more conveniently.
- Improve map-based property search by integrating advanced Google Maps features such as nearby places, distance calculation and location-based recommendations.
- Integrate local payment gateways such as PayHere to support Sri Lankan users more effectively, in addition to Stripe.
- Add multilingual support such as English, Sinhala and Tamil to make the platform easier for local users and tourists.

- Introduce AI-based property recommendations based on user search history, ratings, location and price range.
- Improve the availability calendar with clearer blocked dates, dynamic pricing and seasonal rates.
- Add SMS or WhatsApp notifications for booking confirmations, reminders, cancellations and payment updates.
- Add automated test suites for frontend and backend modules to improve long-term reliability.
- Deploy the system on a cloud platform with CI/CD pipeline, HTTPS, backups and monitoring.
- Improve performance using caching, image optimization and database indexing for large-scale usage.
- Add advanced dispute management and refund policies to handle guest-host issues more professionally.

6.4 Lessons Learnt

The ShortStay project provided practical experience in building a full-stack web application. It helped improve both technical and project-management skills.

- A clear requirement analysis is important before implementation because changes become more difficult after development begins.
- Role-based access control must be planned early when a system has different user types and permissions.
- Separating code into routes, controllers, models, middleware and utilities makes the backend easier to maintain.
- Reusable frontend components such as navigation bars, layouts and API helpers reduce duplicate code and keep the interface consistent.
- Database design is very important for booking systems because users, properties, bookings, payments, reviews and inspections are closely related.

- Testing should start from the early development stage instead of waiting until the full system is completed.
- Secure password handling, token validation and input validation are essential for protecting user data.
- Using Git and GitHub helps the team manage code changes and collaborate more effectively.
- Good documentation helps explain the system clearly to supervisors, users and future developers.
- User feedback is valuable because it highlights interface and workflow issues that developers may not notice during implementation.

In conclusion, the ShortStay system successfully demonstrates the development of a web-based short-term accommodation marketplace. The system provides a strong foundation for connecting guests and hosts while supporting administrative control, payments and property verification. With the suggested future improvements, the system can be developed further into a more complete commercial accommodation booking platform.

References

- Airbnb. (2025). *Airbnb*. <https://www.airbnb.com>
- Booking.com. (2025). *Booking.com*. <https://www.booking.com>
- Figma. (2025). *Figma: The collaborative interface design tool*. <https://www.figma.com>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- GeeksforGeeks. (2025). *Software Engineering Tutorials*. <https://www.geeksforgeeks.org/software-engineering/>
- Google Fonts. (2025). *Google Fonts*. <https://fonts.google.com>
- MySQL. (2025). *MySQL Documentation*. <https://dev.mysql.com/doc/>
- Node.js. (2025). *Node.js Documentation*. <https://nodejs.org>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- React. (2025). *React Documentation*. <https://react.dev>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
- The OpenJS Foundation. (2025). *Node.js*. <https://nodejs.org>
- W3Schools. (2025). *W3Schools Online Web Tutorials*. <https://www.w3schools.com>
- Visual Studio Code. (2025). *Visual Studio Code*. <https://code.visualstudio.com>
- Postman. (2025). *Postman API Platform*. <https://www.postman.com>
- Git. (2025). *Git*. <https://git-scm.com>

Appendix A - System Manual

System requirements

A.1 Introduction

The ShortStay system is a web-based online marketplace developed to connect guests with property owners for short-term accommodation bookings. The system provides a secure and user-friendly platform where guests can search and reserve accommodations, hosts can manage their property listings, administrators can monitor system activities, payment managers can handle financial transactions, and field inspectors can verify listed properties. This manual provides the necessary information to install, configure, and operate the system.

A.2 System Requirements

Hardware Requirements

Table 15 - Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3	Intel Core i5 or above
RAM	4 GB	8 GB or above
Storage	10 GB Free Space	20 GB Free Space
Internet	Broadband Connection	High-Speed Internet

Software Requirements

Table 16 - Software Requirements

Software	Version
Windows	Windows 10/11
Node.js	Latest LTS Version
MySQL Server	Version 8.0 or above
Visual Studio Code	Latest Version
Google Chrome	Latest Version
Git	Latest Version

A.3 Installation Procedure

Follow the steps below to install the ShortStay system.

1. Install Node.js on the computer.
2. Install MySQL Server and MySQL Workbench.
3. Install Visual Studio Code.
4. Clone the ShortStay project from the GitHub repository.
5. Open the project folder using Visual Studio Code.

6. Install all required project dependencies using:

`npm install`

7. Configure the environment variables in the `.env` file.
8. Import the MySQL database into the local server.
9. Start the backend server.

`npm start`

10. Start the frontend application.

`npm run dev`

11. Open the application using the provided local URL in a web browser.

A.4 Database Configuration

The ShortStay system uses MySQL as the relational database management system.

The following information should be configured before running the application.

- Database Name
- Database Username
- Database Password
- Database Host
- Database Port
- JWT Secret Key

A.5 Project Structure

The project is organized into two main sections.

- Frontend
- Backend

The frontend contains user interface components, pages, routing, and reusable components.

The backend contains controllers, routes, middleware, database models, authentication modules, payment processing, and API services.

A.6 Troubleshooting

Common issues that may occur during installation are listed below.

Table 17 - Troubleshooting

Problem	Solution
Database connection failed	Verify MySQL configuration and credentials.
npm install failed	Check internet connection and Node.js installation.
Application does not start	Ensure all required dependencies are installed.
Login unsuccessful	Verify database records and JWT configuration.

Appendix B - Design Documentation

B.1 Introduction

This appendix presents the design documentation used during the development of the ShortStay system. The design documents describe the overall architecture, database structure, object-oriented design, system workflows, and user interface layouts used to implement the proposed solution.

B.2 System Architecture

The ShortStay system follows a three-tier architecture consisting of the Presentation Layer, Business Logic Layer, and Data Layer.

The Presentation Layer provides interfaces for guests, hosts, administrators, payment managers, and field inspectors.

The Business Logic Layer processes user requests, validates system operations, manages authentication, booking management, property verification, and payment processing.

The Data Layer stores all application data in the MySQL database.

B.3 Use Case Diagram

The Use Case Diagram illustrates the interactions between users and the ShortStay system.

The main actors include:

- Guest
- Host
- Administrator
- Payment Manager
- Field Inspector

The major use cases include user registration, login, accommodation search, booking management, payment processing, property verification, report generation, and review management.

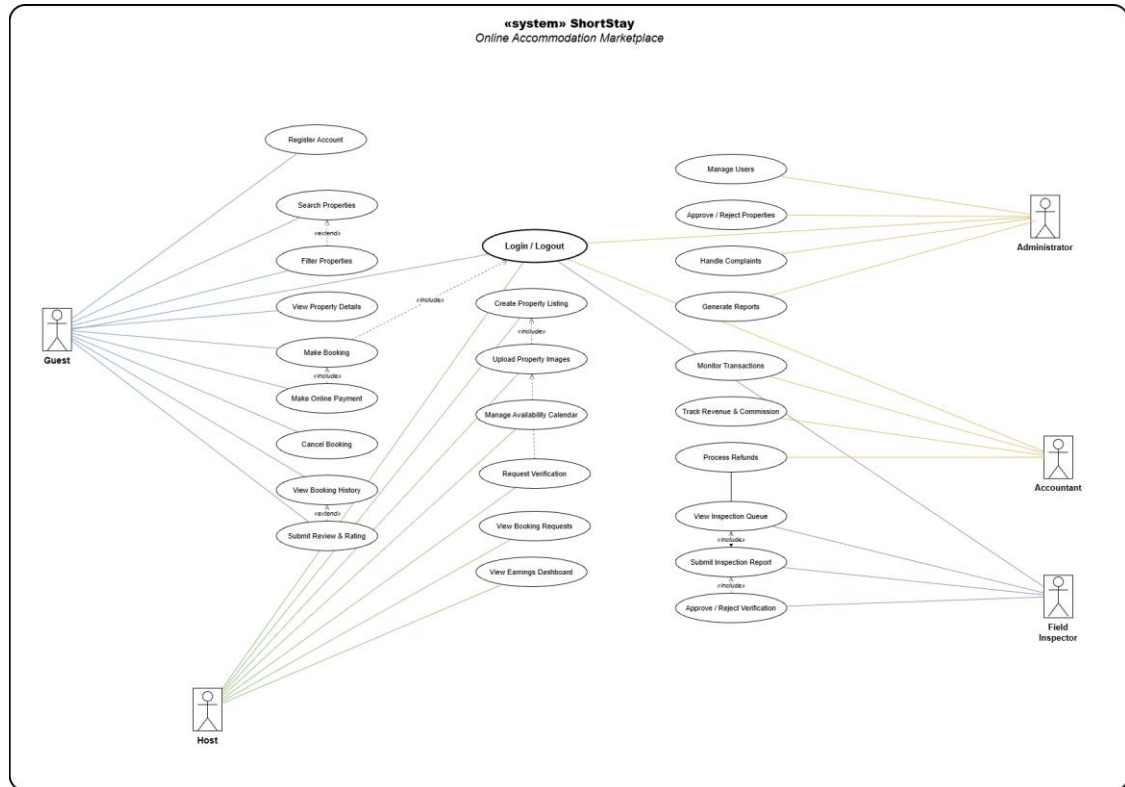


Figure 27 - Use Case Diagram

B.4 Class Diagram

The Class Diagram represents the object-oriented structure of the system, including classes, attributes, methods, and relationships among system entities.

Major classes include:

- User
- Guest
- Host
- Property
- Booking
- Payment

- Review
- Inspection

(Insert Class Diagram Here)

B.5 Entity Relationship Diagram

The ER Diagram illustrates the relationships between the database entities.

Major entities include:

- User
- Host
- Guest
- Staff
- Property
- Booking
- Payment
- Review
- Inspection
- Notification
- Complain

The relationships ensure data integrity and efficient management of accommodation bookings and payments.

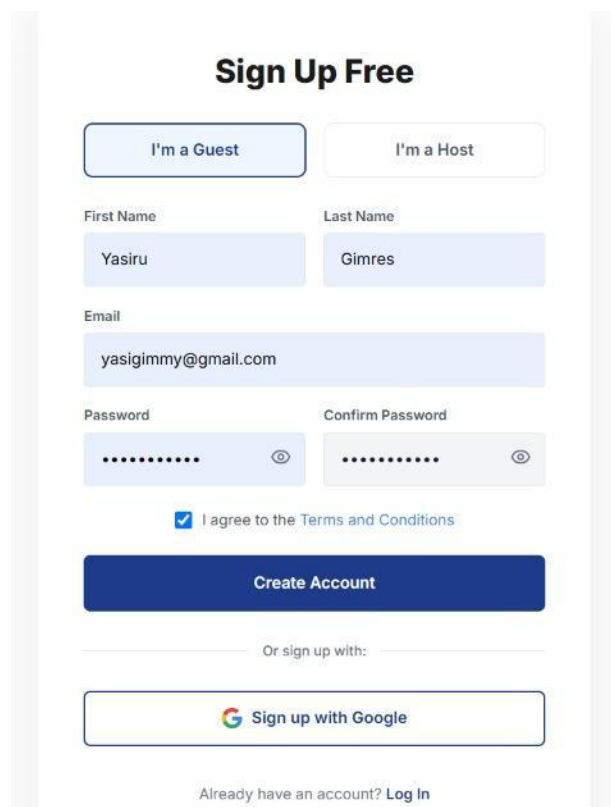
Appendix C - User Manual

C.1 Introduction

This user manual explains how different users can operate the ShortStay system. It provides step-by-step instructions for performing the major functions of the application.

C.2 User Registration

1. Open the ShortStay website.
2. Click the **Register** button.
3. Enter the required personal information.
4. Submit the registration form.
5. The user account will be created successfully.

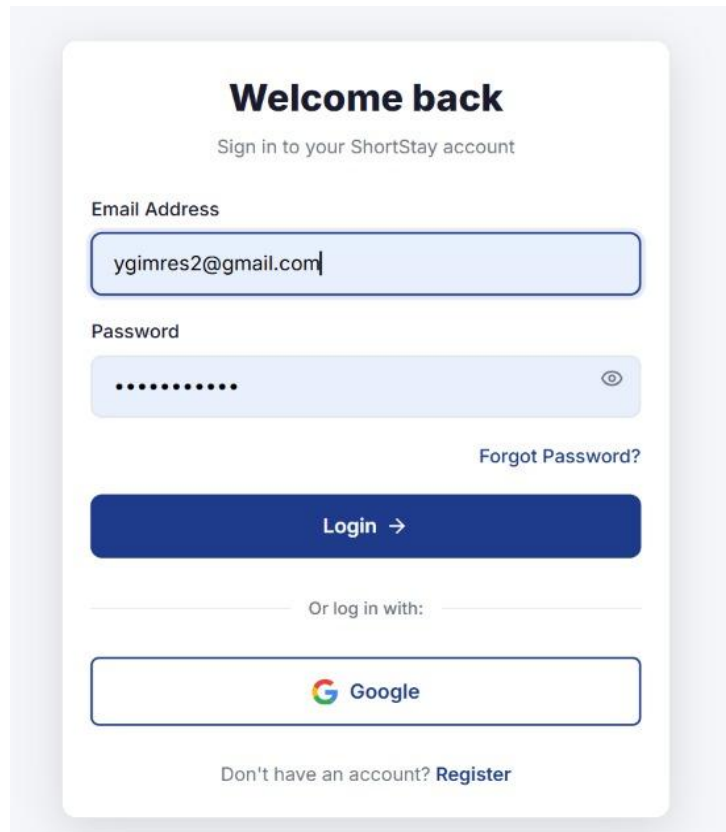


The image shows a 'Sign Up Free' registration form. At the top, there are two buttons: 'I'm a Guest' (highlighted in blue) and 'I'm a Host'. Below these are input fields for 'First Name' (containing 'Yasiru') and 'Last Name' (containing 'Gimres'). The 'Email' field contains 'yasigimmy@gmail.com'. There are two password fields: 'Password' and 'Confirm Password', both masked with dots and having an eye icon to toggle visibility. Below the password fields is a checkbox labeled 'I agree to the Terms and Conditions' which is checked. A large blue 'Create Account' button is positioned below the checkbox. Underneath this button is the text 'Or sign up with:' followed by a 'Sign up with Google' button featuring the Google logo. At the very bottom, there is a link that says 'Already have an account? Log In'.

Figure 29 - UI for User Registration

C.3 User Login

1. Open the Login page.
2. Enter the registered email address.
3. Enter the password.
4. Click the **Login** button.
5. The user will be redirected to the appropriate dashboard.



The image shows a user login interface for 'ShortStay'. At the top, it says 'Welcome back' and 'Sign in to your ShortStay account'. Below this are two input fields: 'Email Address' with the text 'ygimres2@gmail.com' and 'Password' with a masked password '.....'. To the right of the password field is a 'Forgot Password?' link. Below the inputs is a blue 'Login →' button. Underneath the button is a horizontal line with the text 'Or log in with:'. Below this line is a button with the Google logo and the text 'Google'. At the bottom, it says 'Don't have an account? Register'.

Figure 30 - UI for User Login

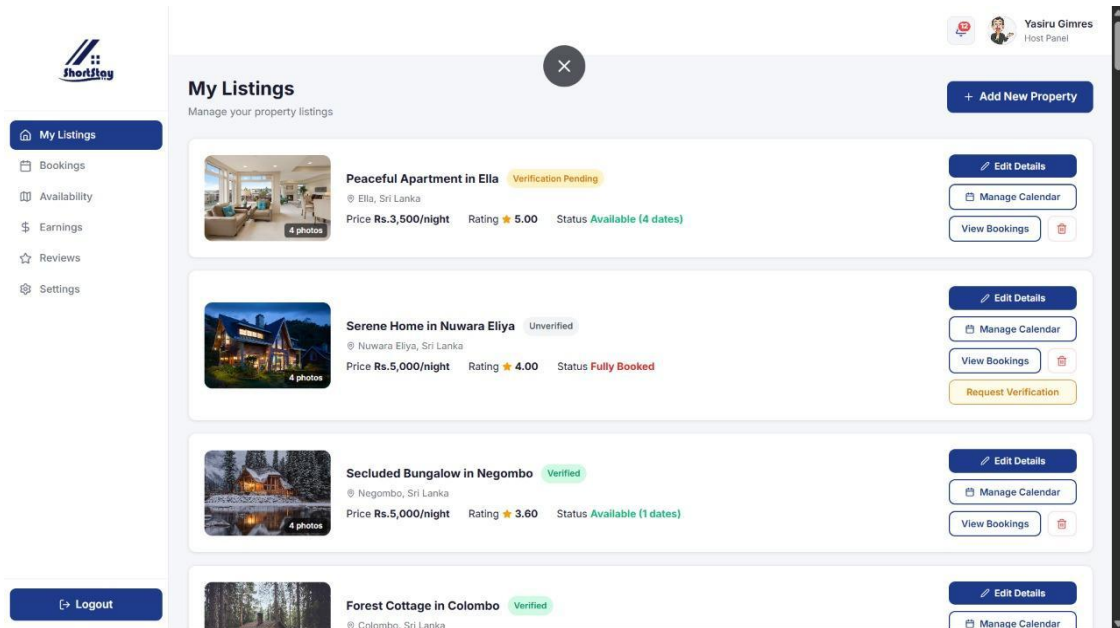


Figure 31 - UI for Host dashboard

C.4 Searching for Properties

1. Navigate to the Browse Properties page.
2. Enter the preferred location or property title.
3. Apply filters such as price, property type, and availability.
4. Select a property to view detailed information.

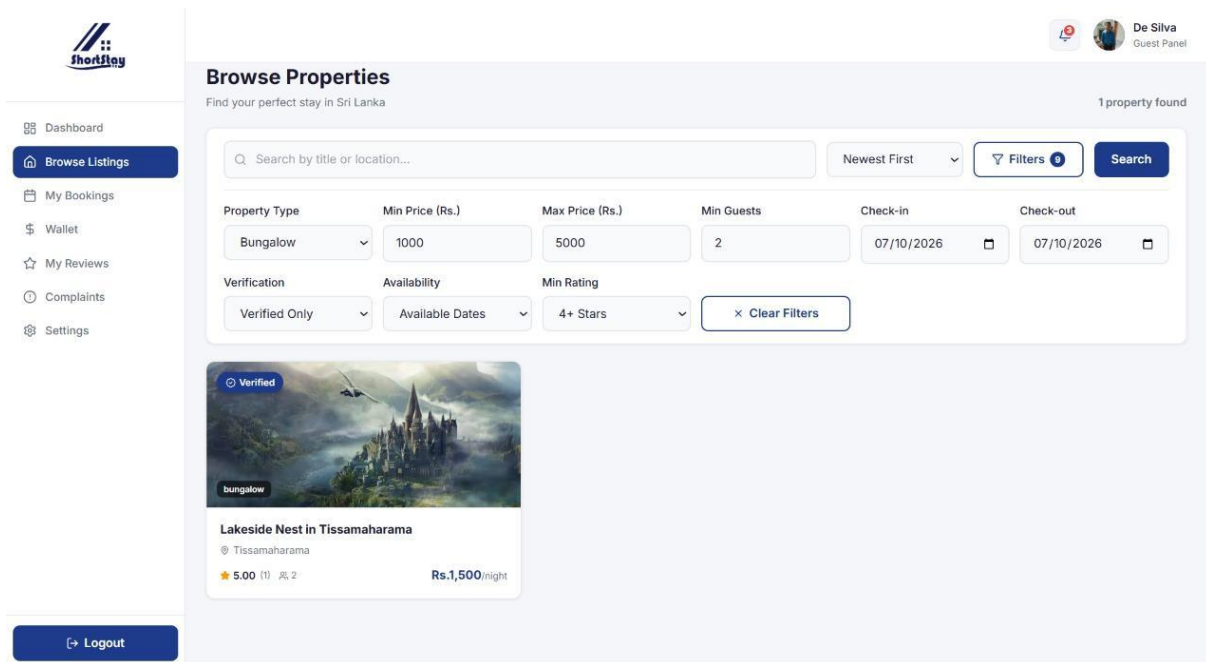


Figure 32 - UI 1 for Searching for Properties

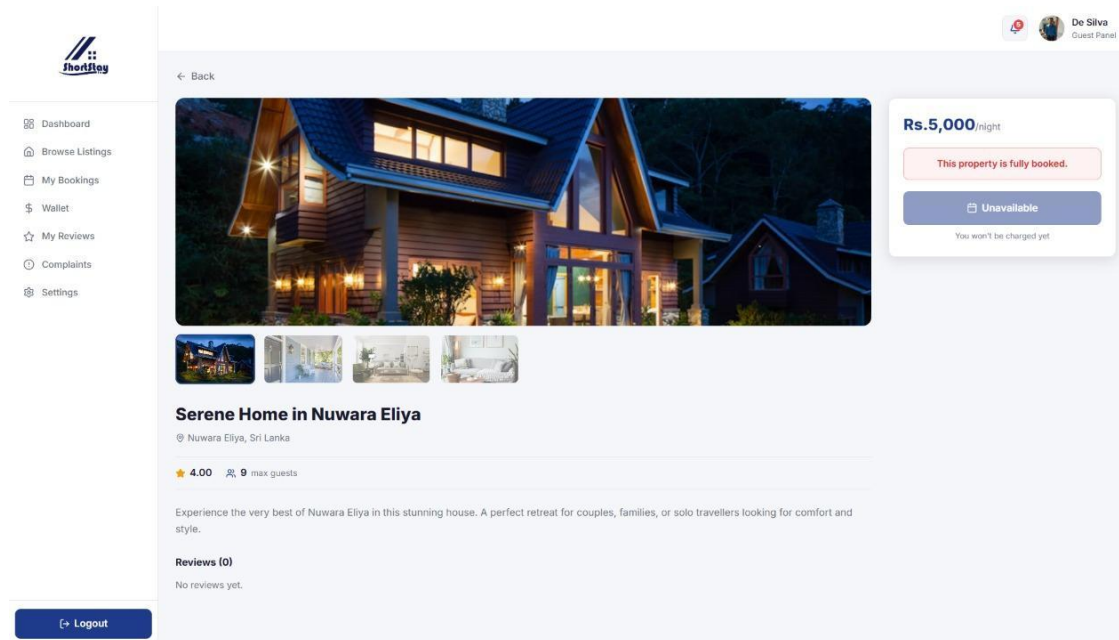


Figure 33 - UI 2 for Searching for Properties

C.5 Booking Accommodation

1. Open the selected property.
2. Select the check-in and check-out dates.
3. Confirm the booking details.
4. Getting booking approval from the host.

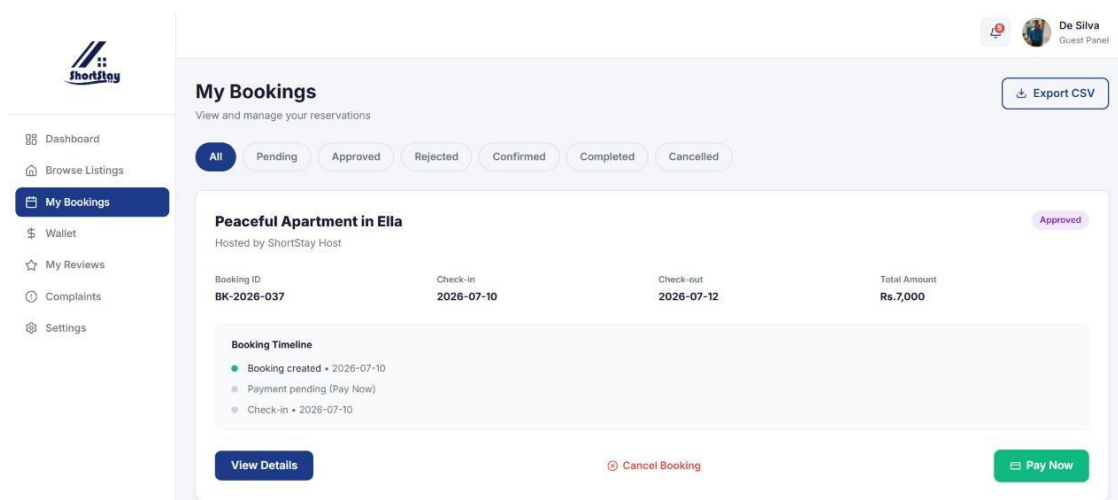


Figure 34 - UI 1 for Booking Accommodation

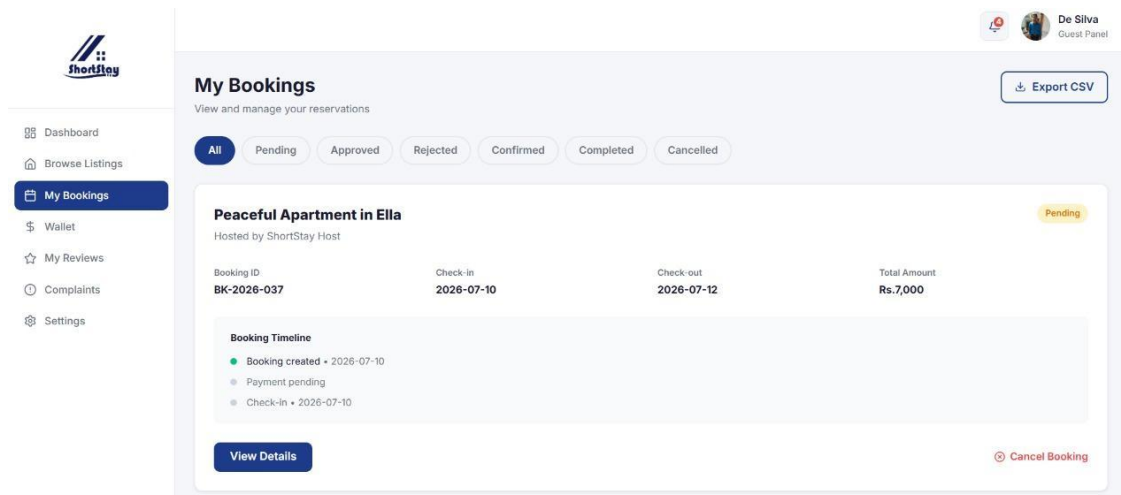


Figure 35 - UI 2 for Booking Accommodation

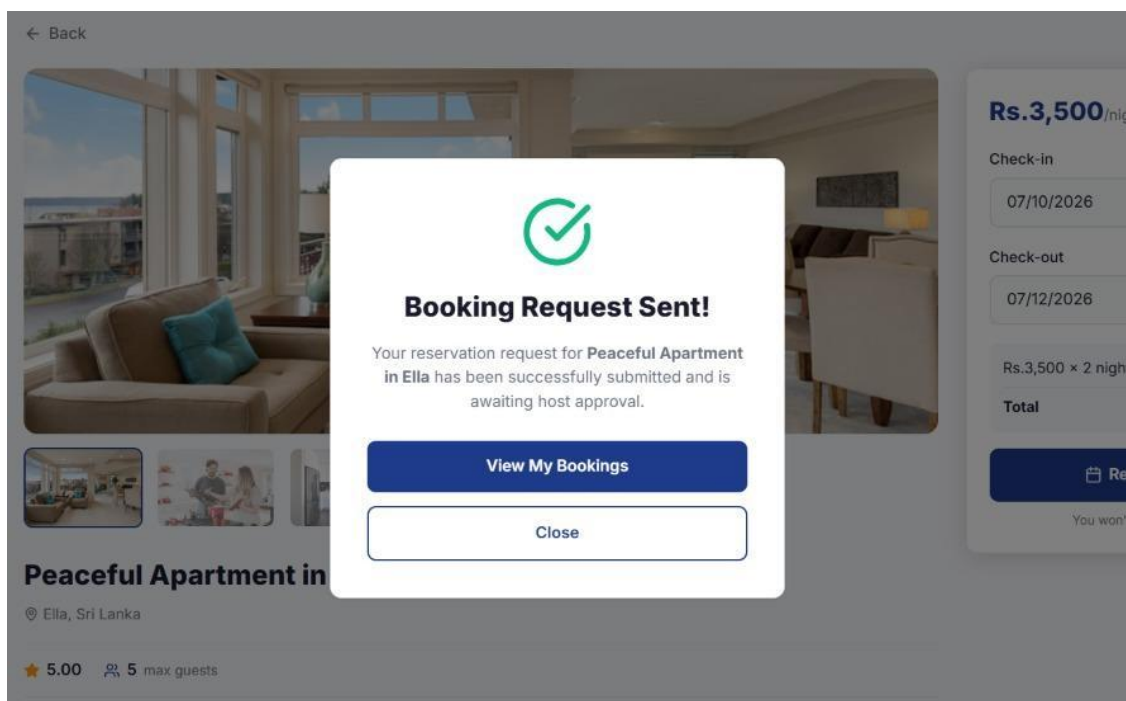


Figure 36 - UI 3 for Booking Accommodation

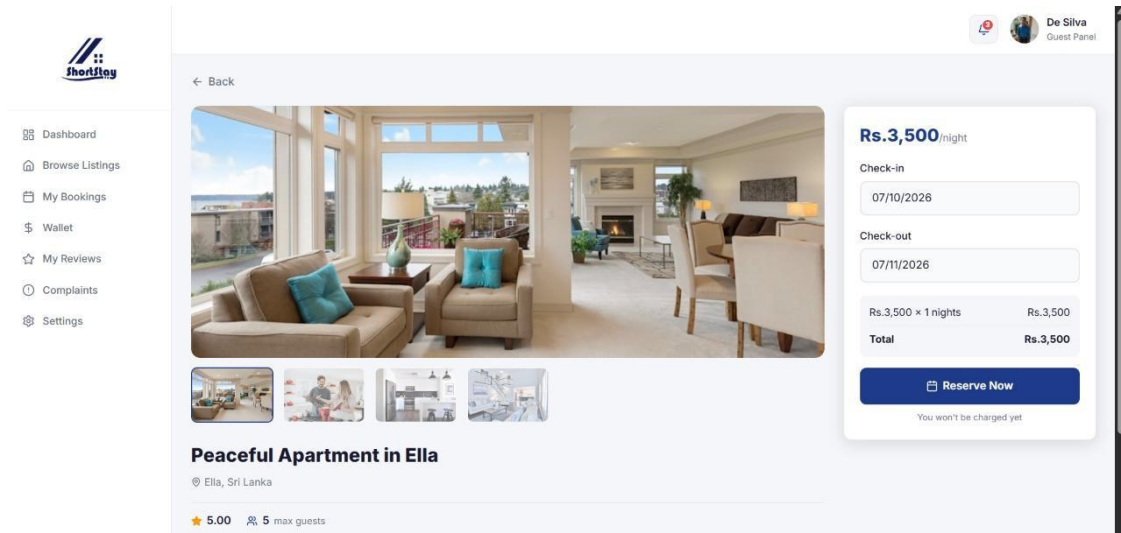


Figure 37 - UI 4 for Booking Accommodation

C.6 Making Payments

1. Proceed to the payment page.
2. Enter the payment details.
3. Confirm the payment.
4. A booking confirmation message will be displayed.

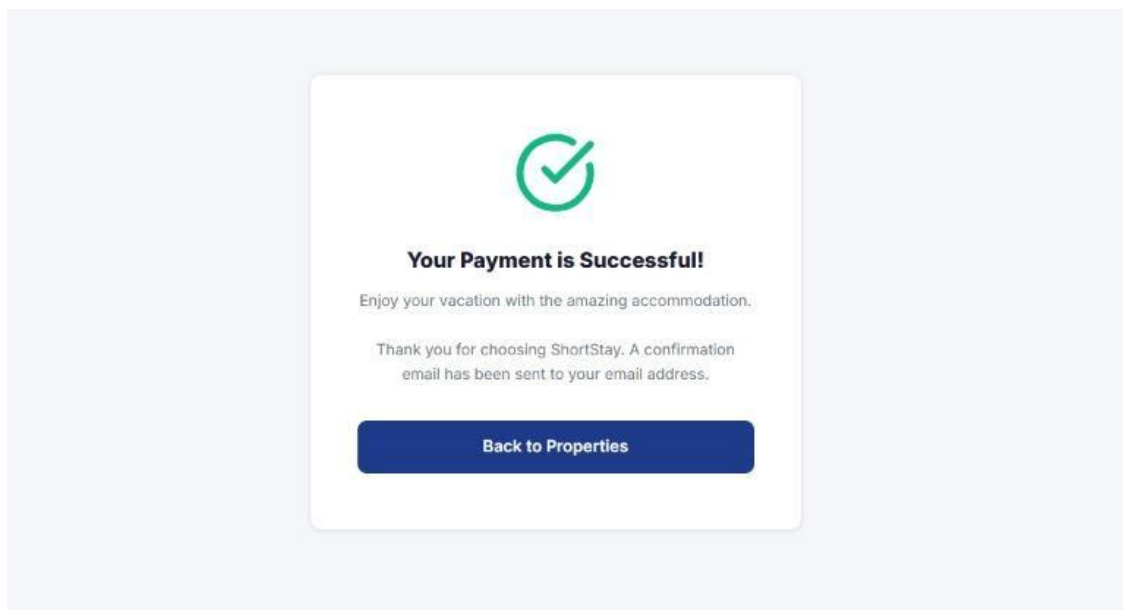


Figure 38 - UI 1 for Making Payments

Shortfly

De Silva Guest Panel

← Back to Property

Billing Payment

Test data filled! You can now click Pay Now.

Payment Type
Credit Card

Credit Card Number
4242 4242 4242 4242

Expiry Date (MM/YEAR)
12/2026

CVV
123

First Name
John

Last Name
Doe

Billing Address
123 Main Street

City
Colombo

Province
Western

Postal Code
00100

Mobile Number
+94 77 123 4567

Email Address
john.doe@email.com

[> Logout]

Cost Breakdown

Number of Days
2

Total cost of rent (2 days)
Rs. 7,000

Discount (1%)
- Rs. 70

Total Charge
Rs. 6,930

Pay Now

Fill Test Data

Simulate Failed Payment

Figure 39 - UI 2 for Making Payments

C.7 Host Functions

The host can perform the following tasks:

- Add new properties
- Update property details
- Manage booking requests
- View earnings
- Track reservation status

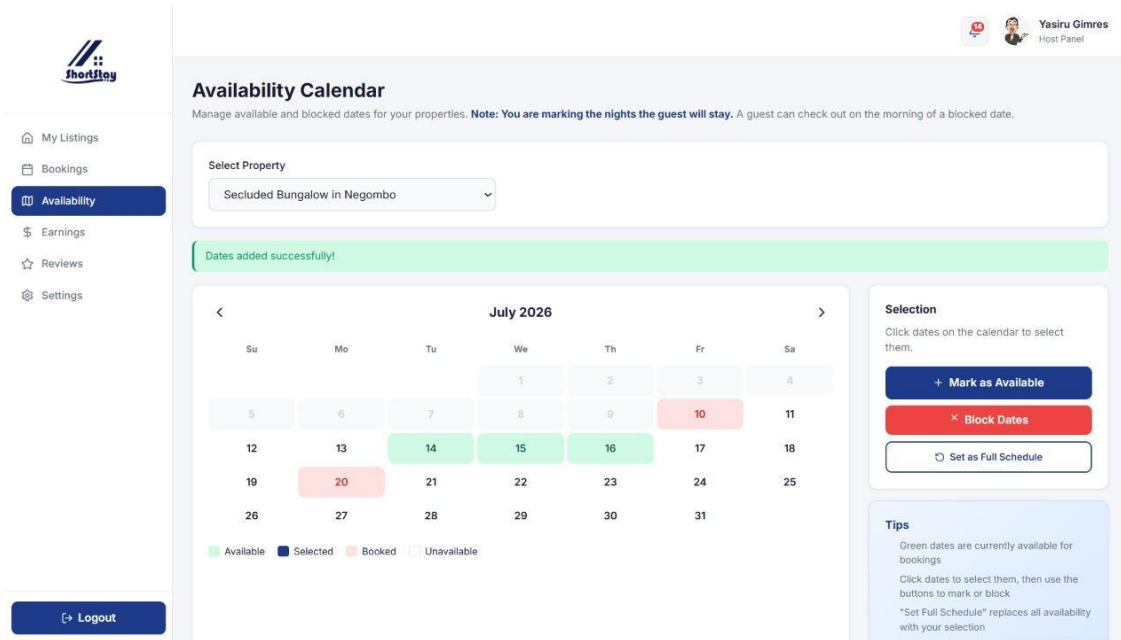


Figure 40 - UI 1 for Host Functions

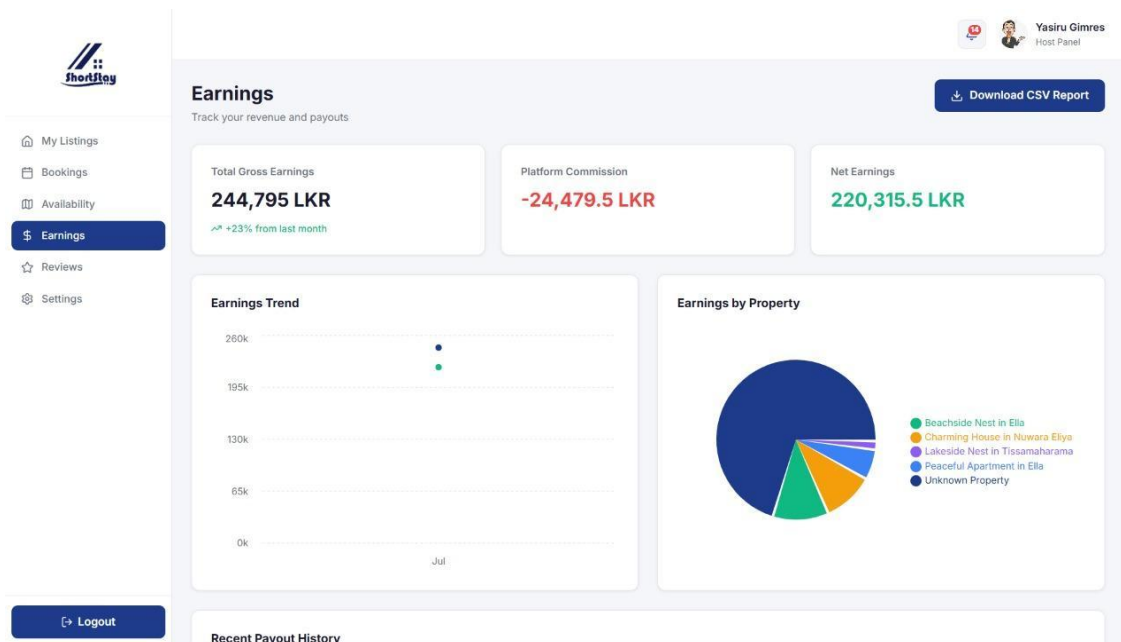


Figure 41 - UI 2 for Host Functions

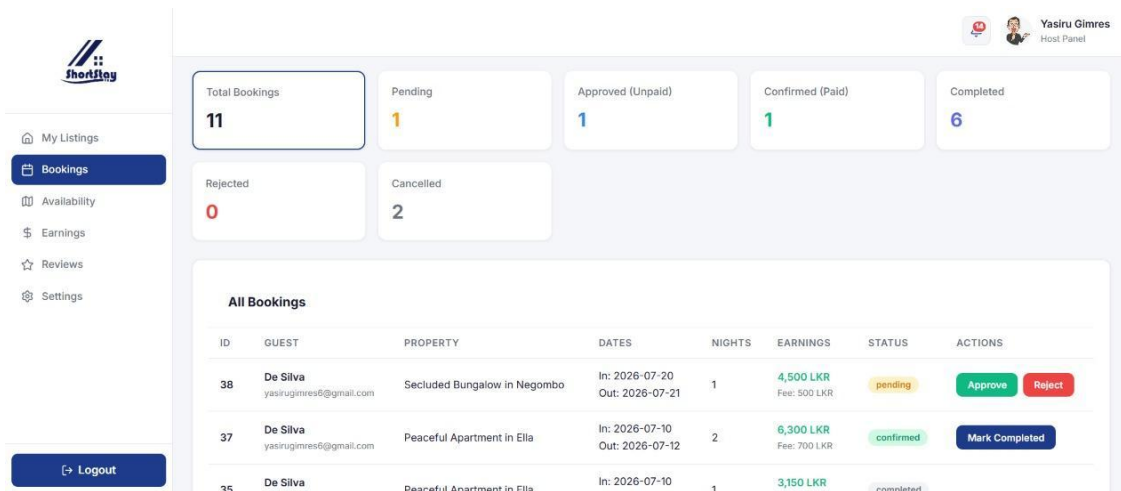


Figure 42 - UI 3 for Host Functions

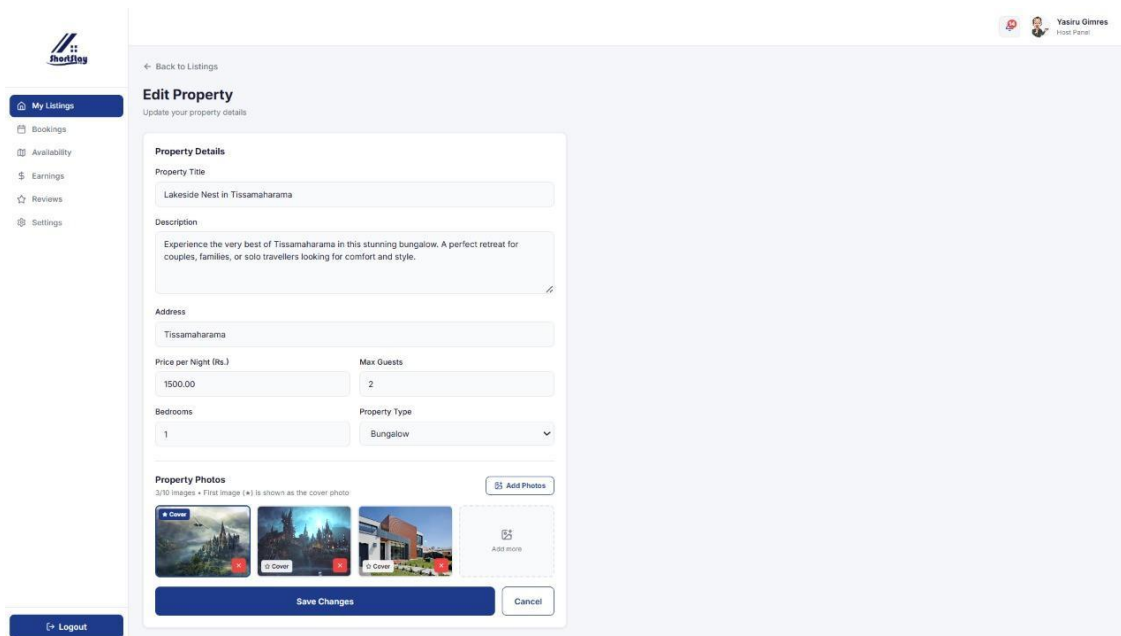


Figure 43 - UI 4 for Host Functions

The screenshot shows the 'Add New Property' form in the ShortStay Host Panel. The form is titled 'Add New Property' and includes a subtitle 'List a new property on ShortStay'. The form fields are as follows:

- Property Details**
 - Property Title**: A text input field with a placeholder 'e.g. Luxury Beachfront Villa in Galle'.
 - Description**: A text area with a placeholder 'Describe your property, amenities, and surroundings...'.
 - Address**: A text input field with a placeholder 'Full address including city'.
 - Price per Night (Rs.)**: A text input field with a value of '5000'.
 - Max Guests**: A text input field with a value of '4'.
 - Bedrooms**: A text input field with a value of '1'.
 - Property Type**: A dropdown menu with 'Apartment' selected.
- Property Photos**: A section with a note '0/10 images • First image (*) is shown as the cover photo' and an 'Add Photos' button. Below this is a large dashed box with an upload icon and the text 'Click to upload property photos' and 'JPG or PNG • Max 5MB each • Up to 10 photos'.

The form has a 'Create Property' button and a 'Cancel' button. The sidebar on the left includes links for 'My Listings', 'Bookings', 'Availability', 'Earnings', 'Reviews', and 'Settings'. The top header shows the user's name 'Yasiru Ginnes' and role 'Host Panel'.

Figure 44 - UI 5 for Host Functions

C.8 Administrator Functions

The administrator is responsible for:

- Managing users
- Approving properties
- Monitoring system activities
- Managing complaints
- Generating reports

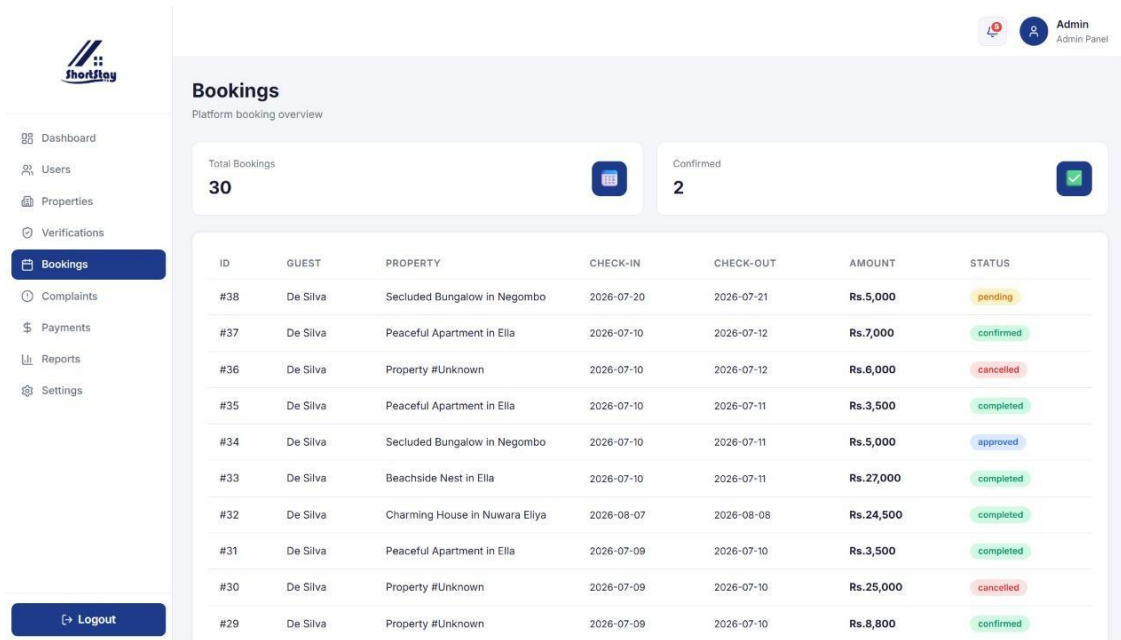


Figure 45 - UI 1 for Administrator Functions

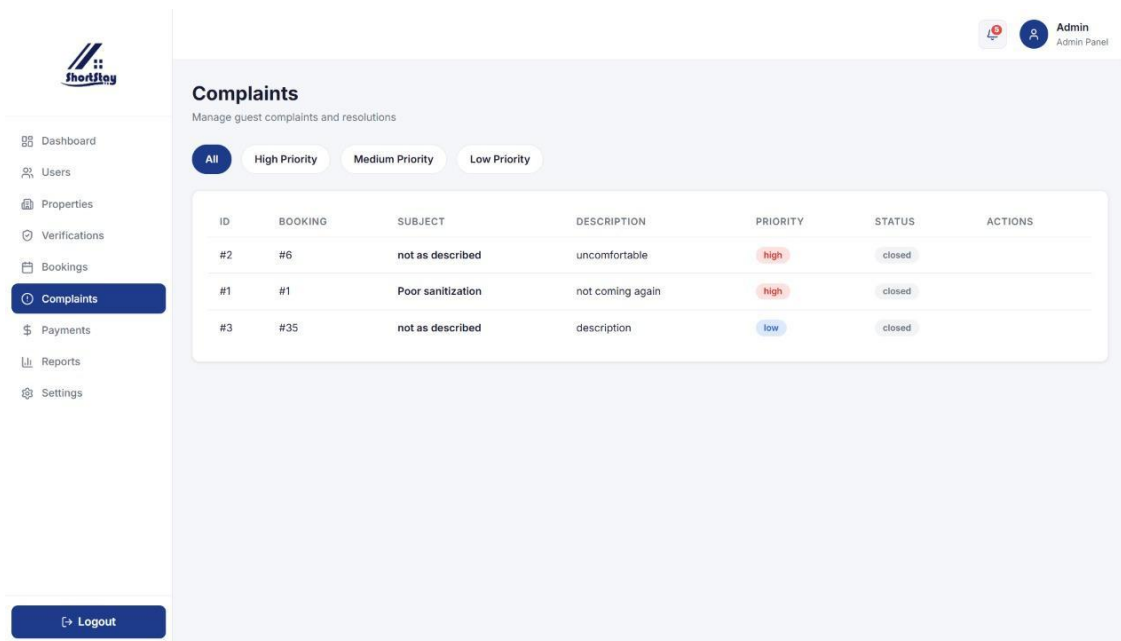


Figure 46 - UI 2 for Administrator Functions

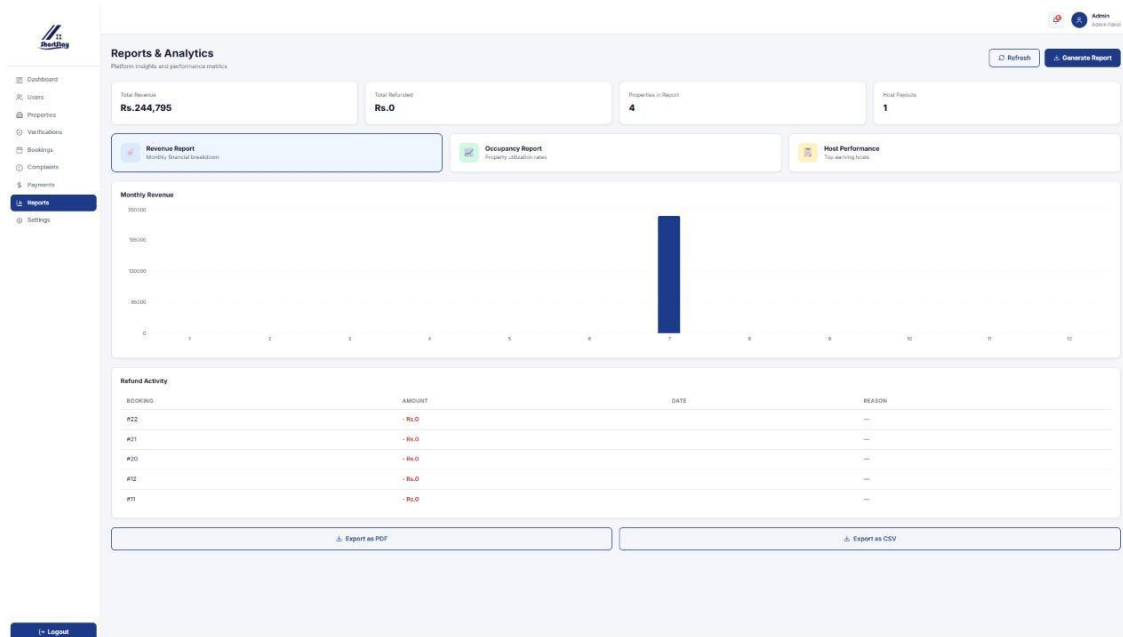


Figure 47 - UI 3 for Administrator Functions

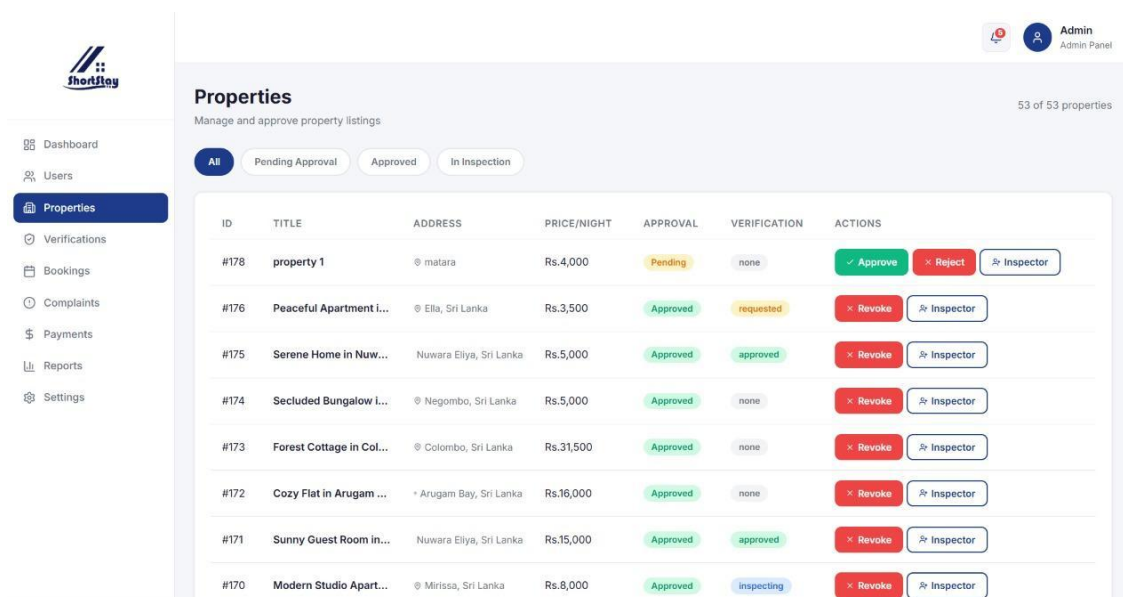


Figure 48 - UI 4 for Administrator Functions

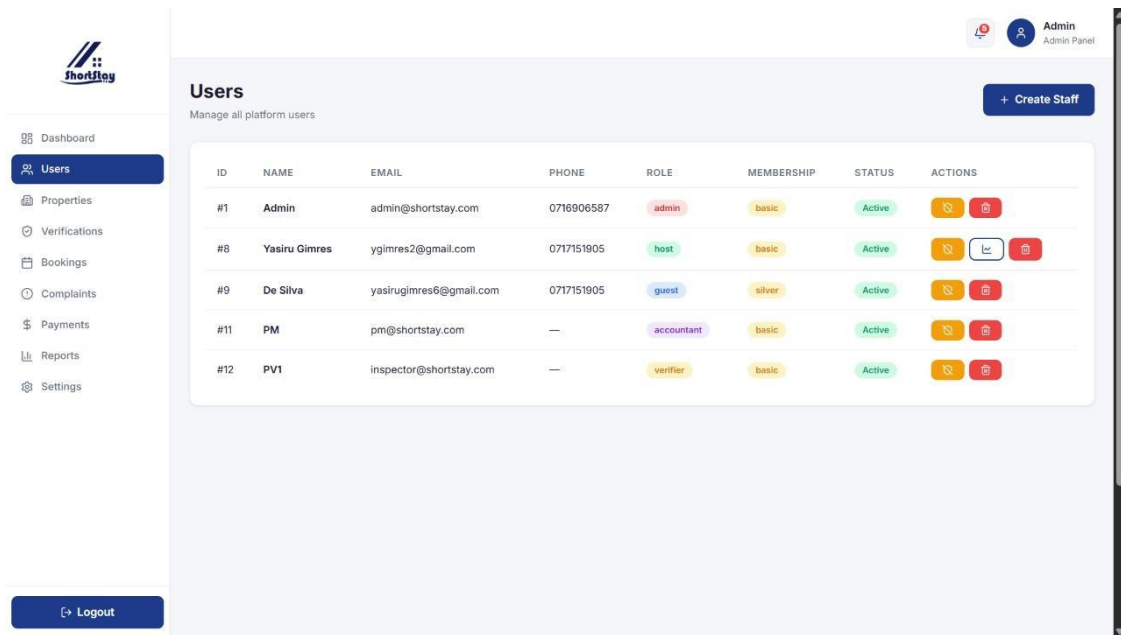


Figure 49 - UI 5 for Administrator Functions

C.9 Payment Manager Functions

The Payment Manager can:

- View payment transactions
- Process refunds
- Generate payment reports
- Monitor financial records

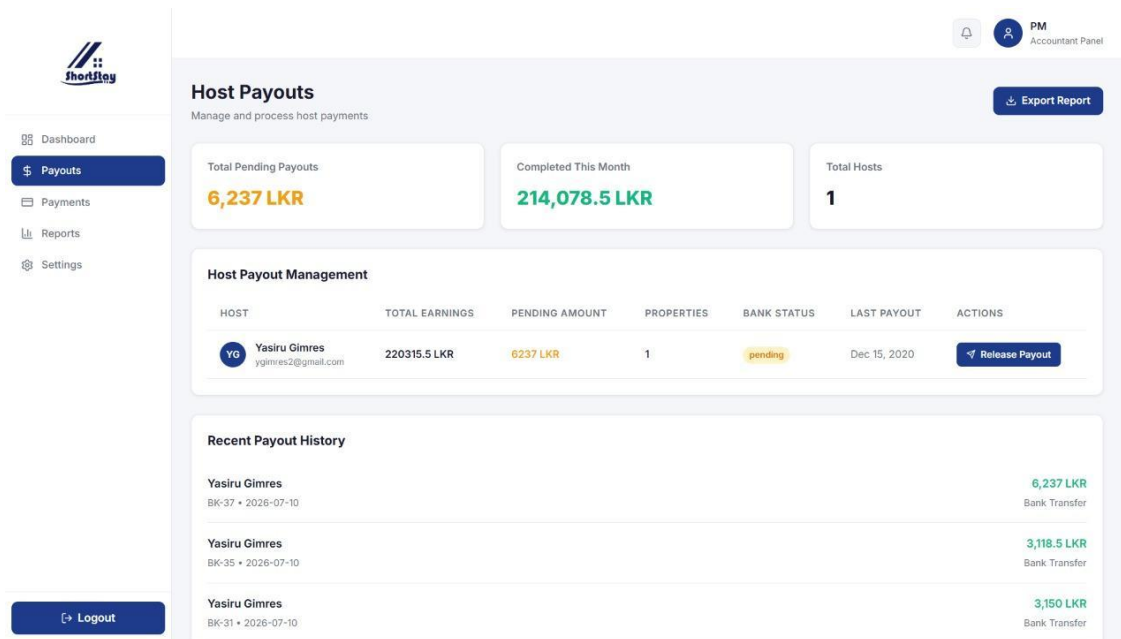


Figure 50 - UI 1 for Payment Manager Functions

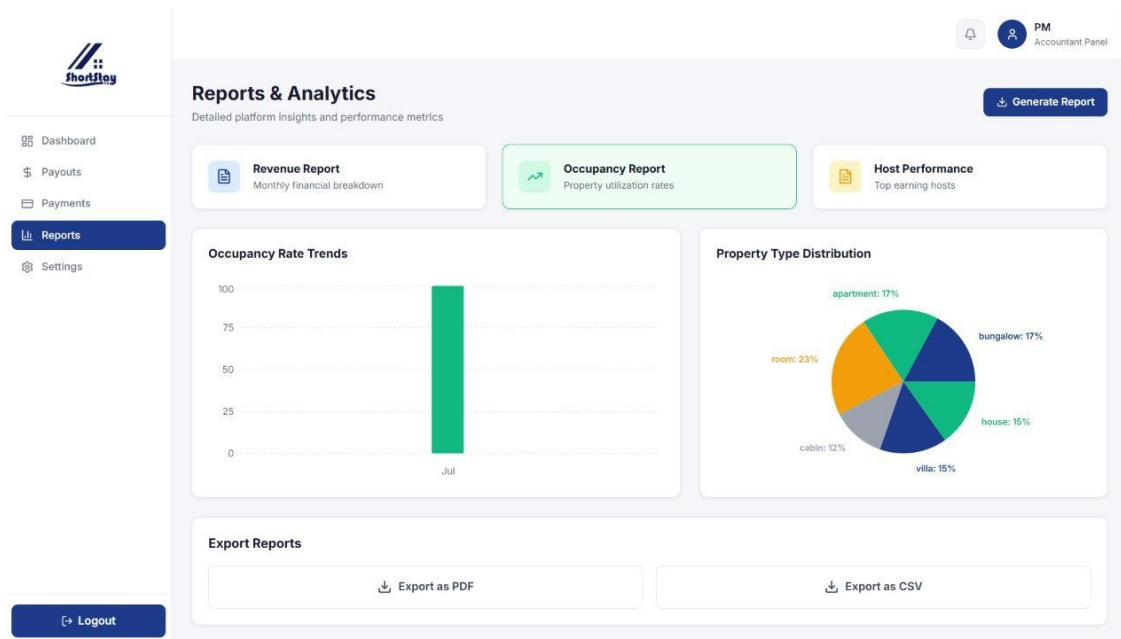


Figure 51 - UI 2 for Payment Manager Functions

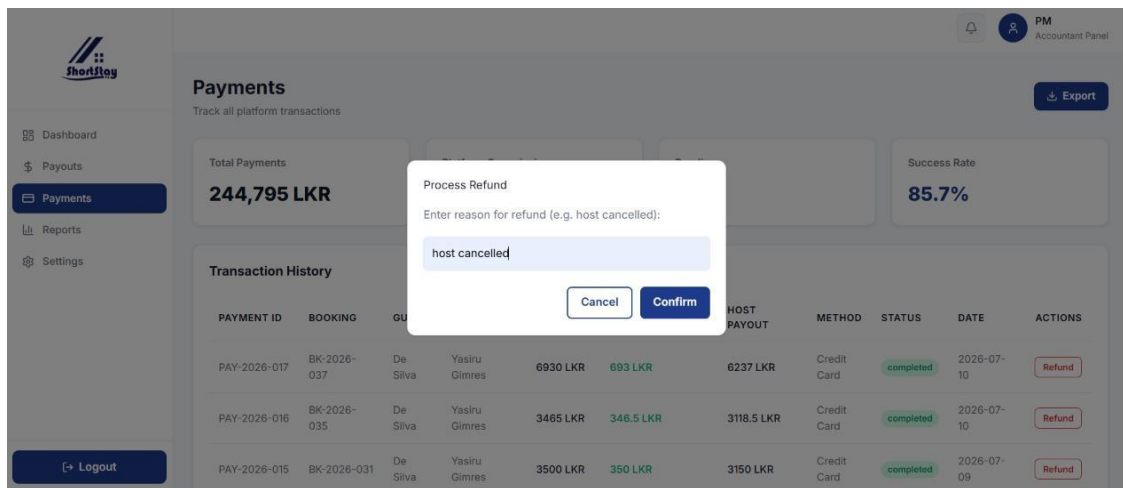


Figure 52 - UI 3 for Payment Manager Functions

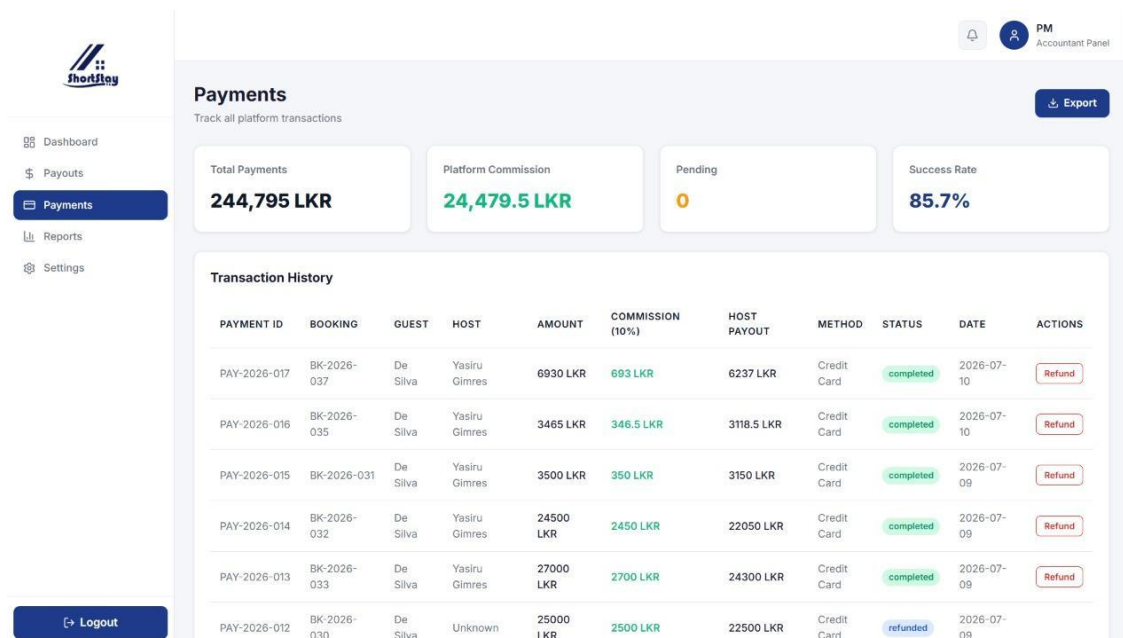


Figure 53 - UI 4 for Payment Manager Functions

C.10 Field Inspector Functions

The Field Inspector is responsible for:

- Viewing inspection requests
- Conducting property inspections
- Approving or rejecting properties
- Updating inspection records

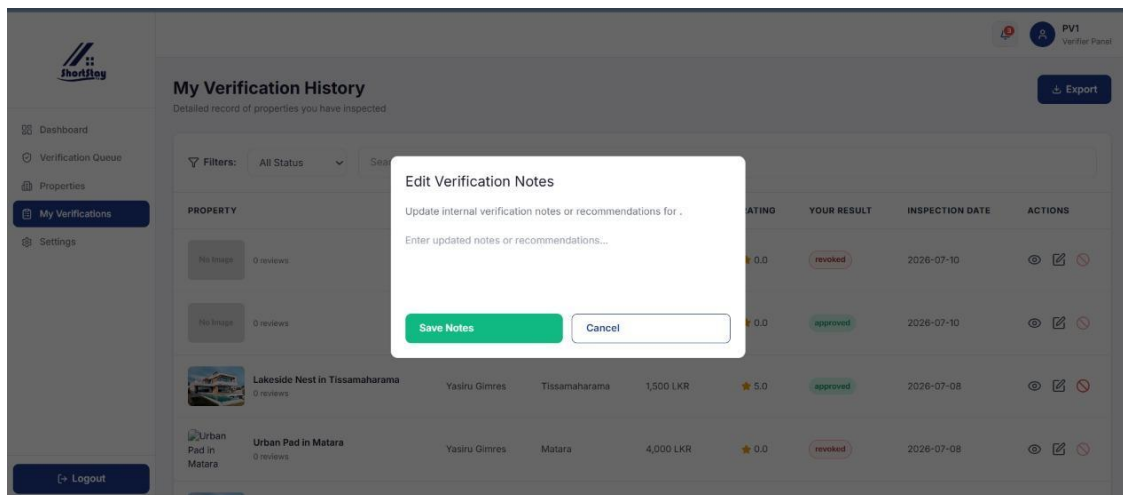


Figure 54 - UI 1 for Field Inspector Functions

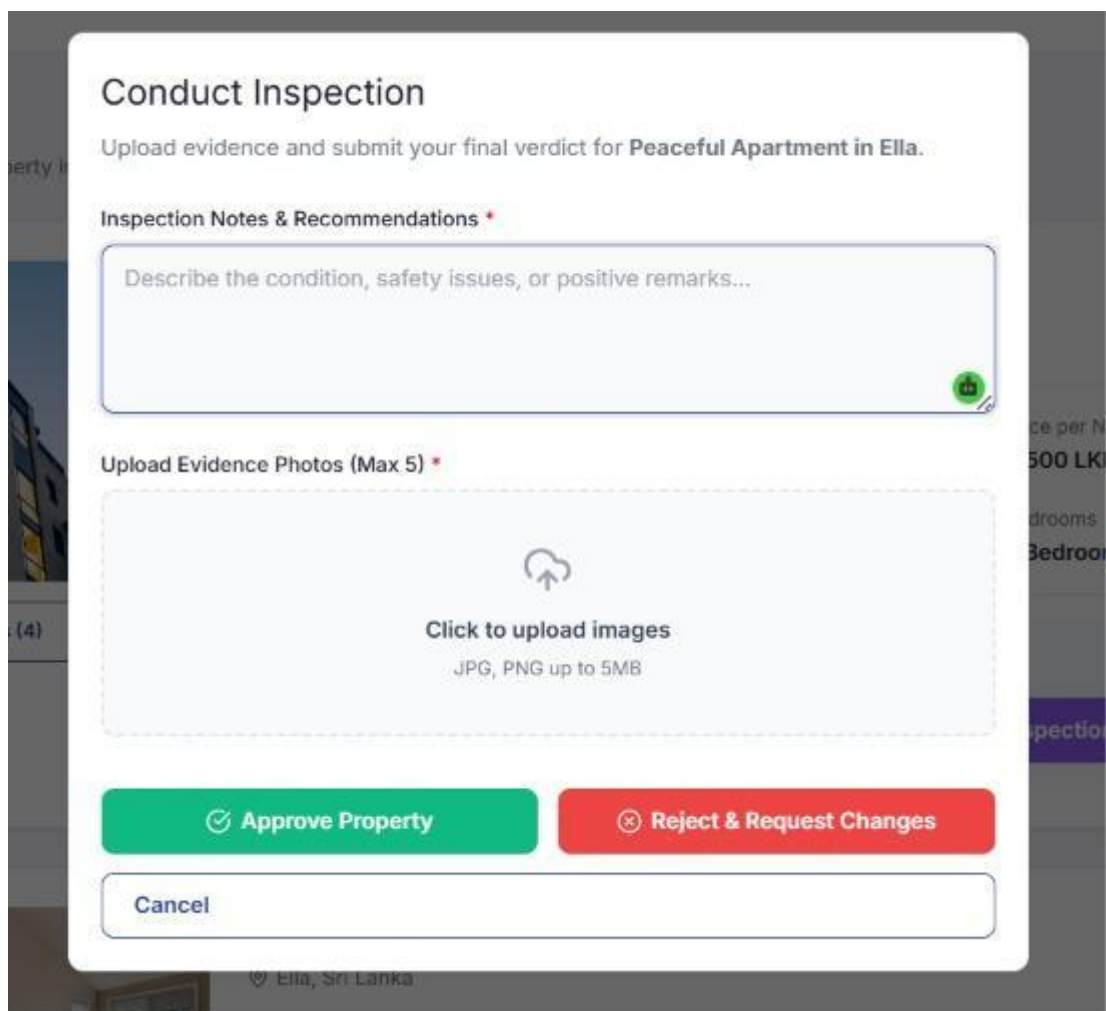


Figure 55 - UI 2 for Field Inspector Functions

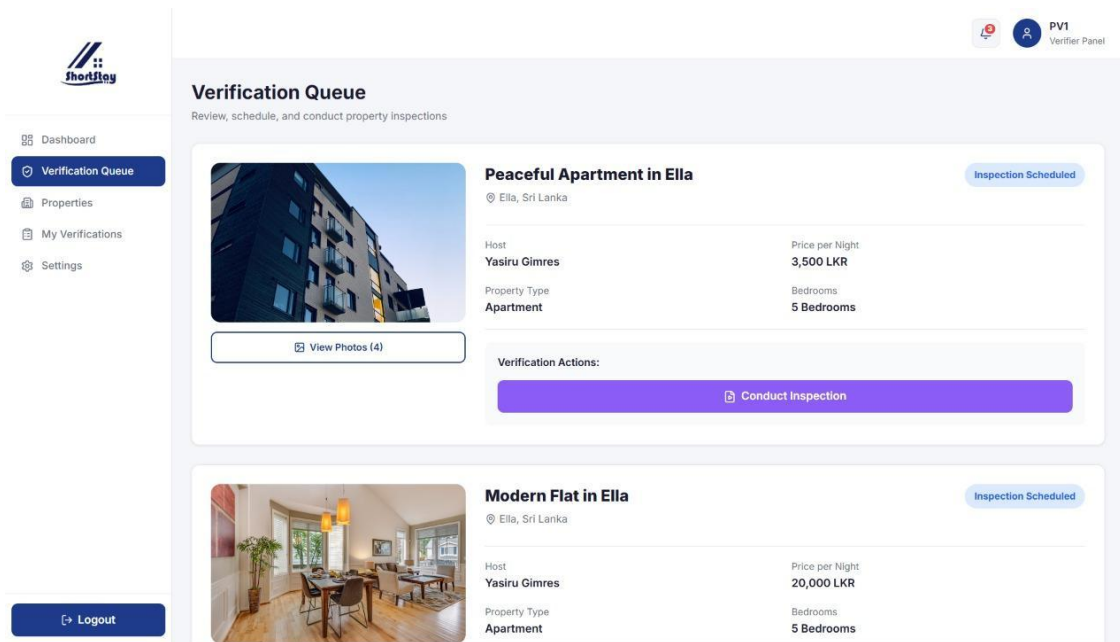


Figure 56 - UI 3 for Field Inspector Functions

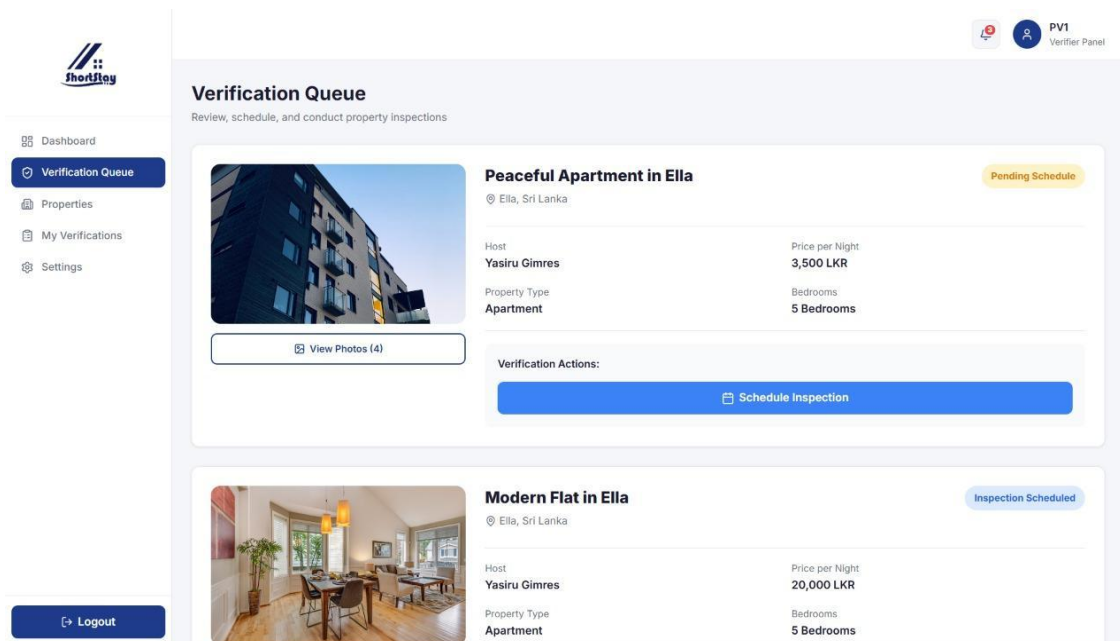


Figure 57 - UI 4 for Field Inspector Functions

C.11 User Logout

1. Click the **Logout** icon.
2. The system will terminate the current session and redirect the user to the login page.

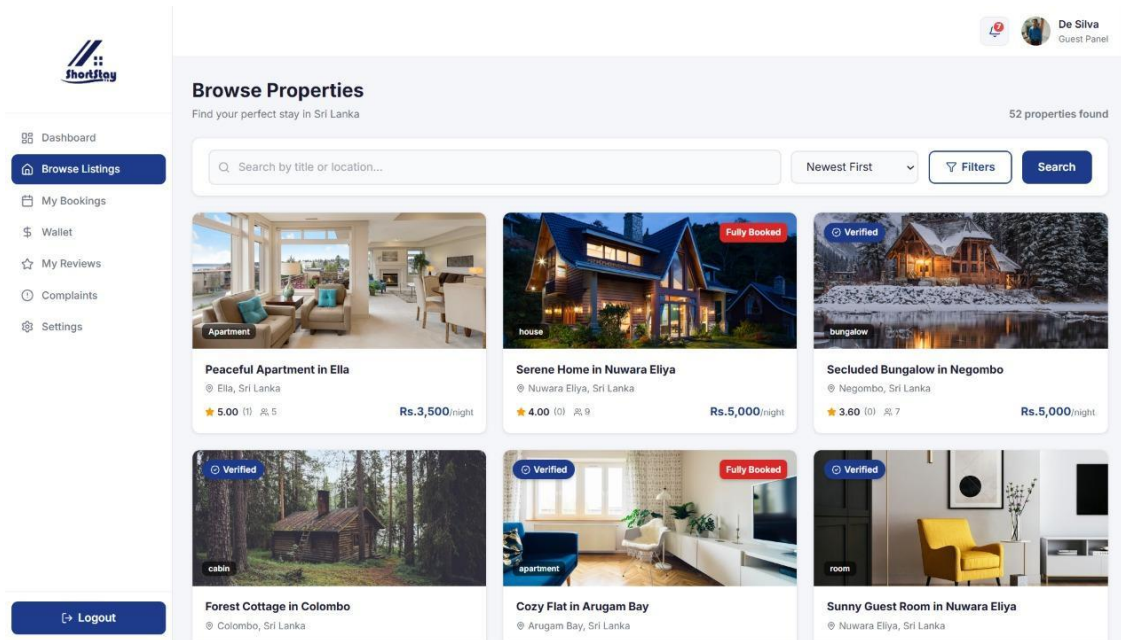


Figure 58 - UI 1 for User Logout

Welcome back

Sign in to your ShortStay account

Email Address

Enter email address

Password

Enter password

[Forgot Password?](#)

Login →

Or log in with:

 Google

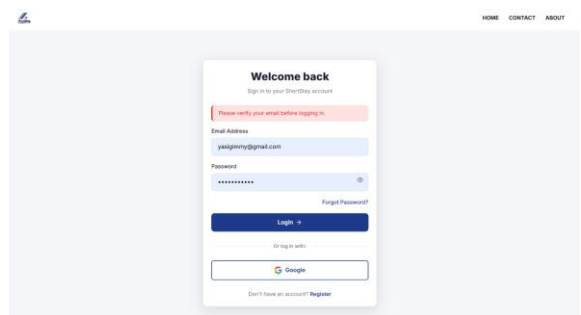
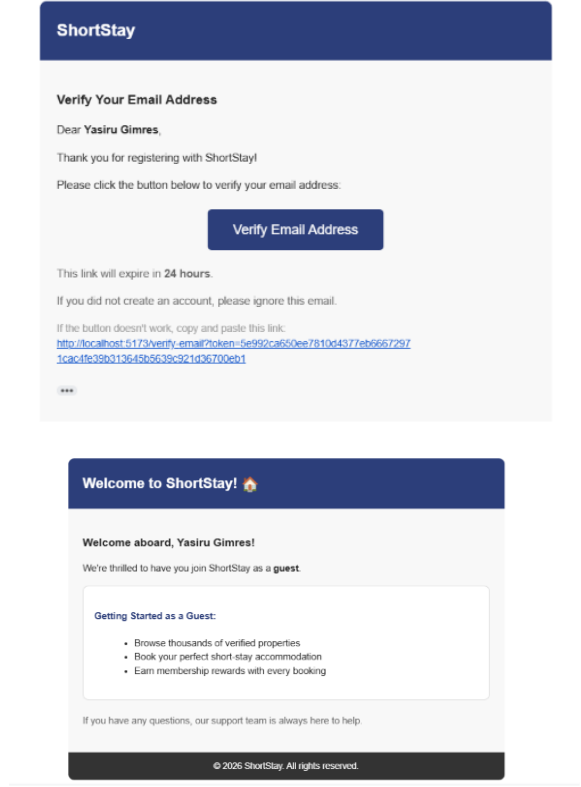
Don't have an account? [Register](#)

Figure 59 - UI 2 for User Logout

Appendix D - Test Results

Test result for login and logout of the system

Table 18 - Test Results for Authentication

Test Case ID	Expected Result	Actual Result
TC- LOG- 003	User should not be allowed to log in and should be prompted to verify the email	
TC- REG 001	User account should be created successfully and a verification email should be sent	

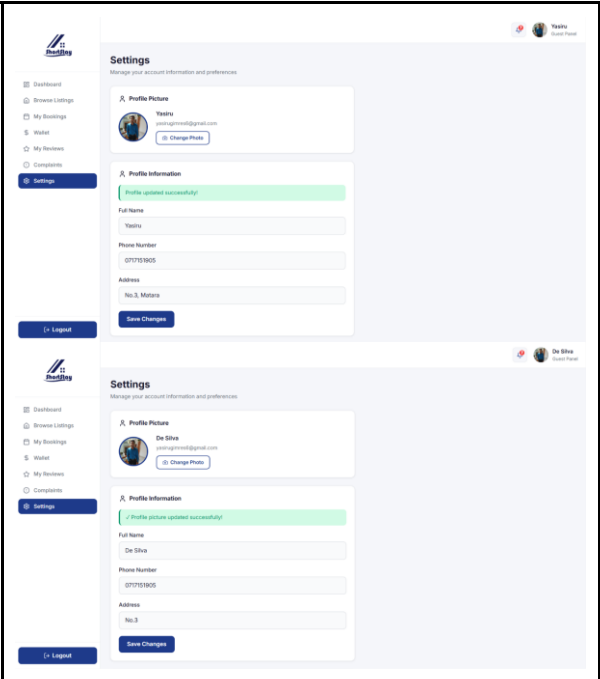
TC- UP- 002 TC- UP- 003	Profile information should be updated successfully. Profile picture should be uploaded and displayed.	
------------------------------------	---	--

Table 19 - Test Results for Property and Search Management

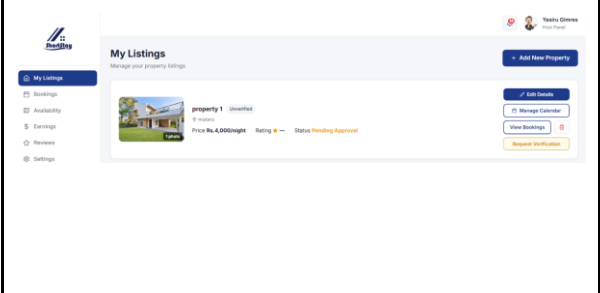
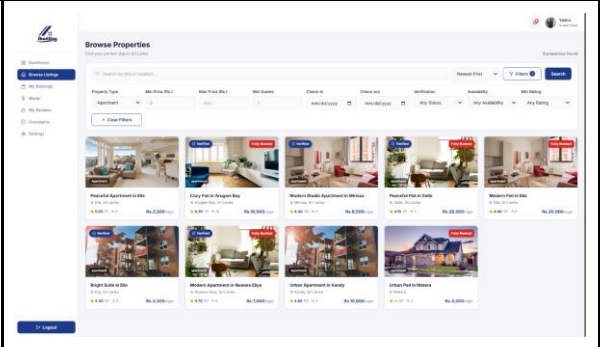
Test Case ID	Expected Result	Actual Result
TC HST 001	Property should be created with Pending Approval status	
TC- SRH-003	Only apartment listings should be displayed	

Table 20 - Test Results for Booking Management

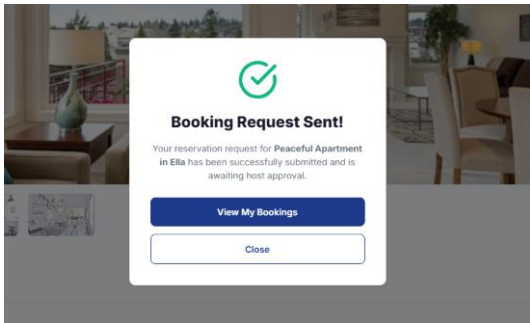
Test Case ID	Expected Result	Actual Result
TC- BKG- 001	Booking should be created successfully	

Table 21 - Test Results for Payment, Reports and Feedback

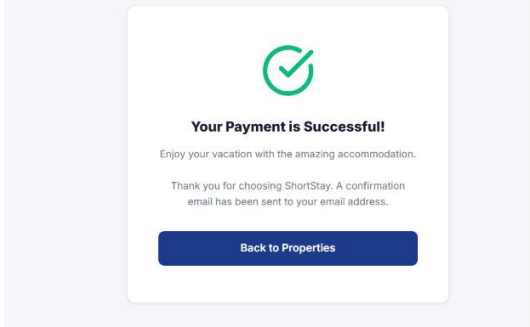
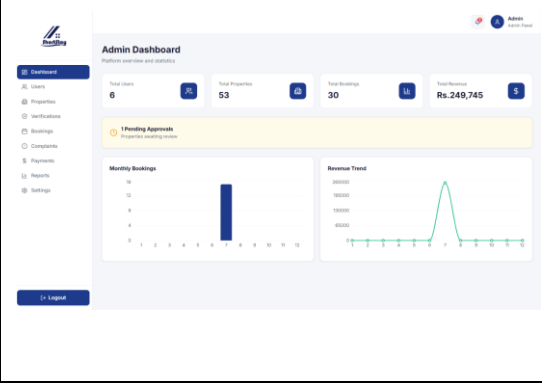
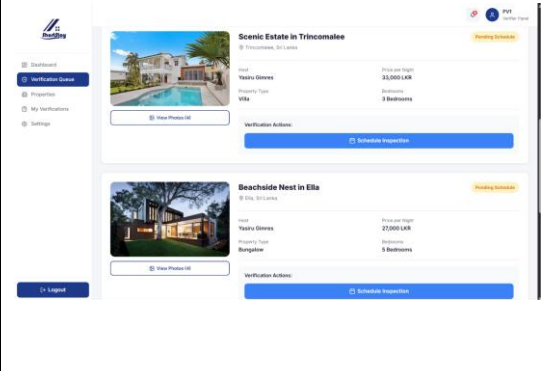
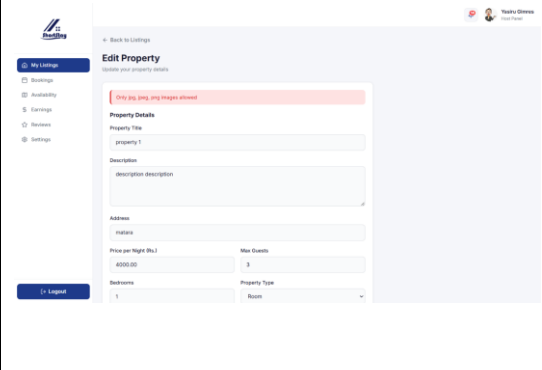
Test Case ID	Expected Result	Actual Result
TC- PAY- 001	Payment should be processed and booking marked as Confirmed	

Table 22 - Test Results for Inspection, Admin and Security

Test Case ID	Expected Result	Actual Result
TC- ADM- 008	Dashboard should display user count, bookings, revenue, and pending verifications.	
TC- INS- 001	Pending property verification requests should be displayed.	
TC- SEC- 004	File upload should be rejected.	

Glossary

Term	Definition
Accommodation	A property available for short-term rental through the ShortStay platform.
Accountant	A system user responsible for monitoring payments, refunds, commissions, and financial records.
Admin	A user who manages the overall operation of the ShortStay system, including users, properties, complaints, and reports.
Authentication	The process of verifying a user's identity before granting system access.
Authorization	The process of controlling user access based on assigned roles and permissions.
Booking	A reservation made by a guest for a selected accommodation.
Dashboard	A role-specific interface displaying relevant information and functions for each user.
Frontend	The client-side part of the application that users interact with through the web browser.
Backend	The server-side application responsible for processing requests, business logic, and database communication.
Guest	A registered user who searches, books, pays for accommodations, and submits reviews.
Host	A registered property owner who lists and manages accommodations.
Inspection	The process of checking a property against required standards before verification.
JWT	A secure authentication token used to validate user sessions.
Module	A functional component of the system such as booking management or payment management.
Payment Transaction	The financial process of collecting guest payments and managing host payouts.
Property Listing	Information published by a host describing an accommodation available for booking.
React.js	The JavaScript library used to develop the frontend of ShortStay.
Review	Feedback and rating submitted by a guest after completing a stay.
Role-Based Access Control	A security mechanism that grants system permissions according to user roles.
ShortStay	The web-based marketplace developed for managing short-term accommodation bookings.
Three-Tier Architecture	The system architecture separating the Presentation Layer, Business Logic Layer, and Data Layer.
Verifier	A system user responsible for inspecting properties and approving or rejecting verification requests.

Index

Term	Page(s)
Accountant	16
Admin	17
Agile Development	26
Backend	27
Booking Management	18
Class Diagram	18,39
Database Design	30,78
Entity Relationship Diagram	18,37
Evaluation	65
Frontend	27
Functional Requirements	24
Guest	16
Host	16
MySQL	32
Node.js	32
Testing	18,65
User Interface	18,41
Verifier	16

Group No: 02				
S.No.	Student Name	Registration Number	Index Number	Email ID
1.	Jayaweera K.U.E	FC110620	FC213008	kavindujayaweera624@gmail.com

2.	Nishanka N.M.R.S	FC110630	FC213018	shashimalravindu2287@gmail.com
3.	Dinushanth S.	FC110661	FC213034	sirchillzv@gmail.com
4.	Wickramasinghe W.M.D.D	FC110641	FC213029	disiniwickramasinghe@gmail.com
5.	Gunasekara M.P.I.S	FC110615	FC213003	indurasachi@gmail.com
6.	De Silva W.P.Y.G	FC110986	FC213039	ygimres2@gmail.com
7.	Zeenathsharoth M.Y	FC110625	FC213013	zeenathsharoth51@gmail.com
8.	Nizmy M.N.M	FC110635	FC213023	Nizmynisthar7788@gmail.com